
Nemsys, una plataforma web colaborativa para compartir y acceder a recursos académicos

Nemsys, a collaborative web platform for sharing and accessing academic resources



Trabajo de Fin de Grado

Curso 2025–2026

Autor

Marco Antonio Pérez Neira

Director

Ramón González del Campo Rodríguez Barbero

Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid

Resumen

Nemtsy, una plataforma web colaborativa para compartir y acceder a recursos académicos

Hasta hace relativamente poco, en España no existían plataformas bien integradas donde compartir apuntes, exámenes u otros recursos académicos; los estudiantes dependían de intercambios en persona o de grupos de mensajería sin persistencia real. En 2015 se lanzó Wuolah, una plataforma que sí se integraba con los centros educativos y permitía subir apuntes vinculados específicamente a una asignatura de un grado concreto. Sin embargo, esta plataforma presenta limitaciones significativas de rendimiento y un modelo de monetización basado en publicidad intensiva que deteriora la experiencia del usuario.

Este Trabajo de Fin de Grado describe el diseño e implementación de una aplicación web alternativa que permite a estudiantes compartir, buscar y descargar apuntes sin interferencias, con un énfasis en la rapidez y la ligereza desde el inicio. La plataforma está diseñada para integrarse con las asignaturas, estudios y centros educativos tanto de España como del extranjero. Debido a las limitaciones de tiempo y al carácter tedioso del proceso más que dificultad técnica, la integración completa solo se ha realizado sobre la UCM, aunque la arquitectura está preparada para su expansión a otros centros: únicamente sería necesario desarrollar un *web crawler* adaptado a la página de cada uno.

Para ello se ha utilizado una combinación de *web scraping* y el desarrollo de un *script* que utiliza JetBrains SWOT, una lista de dominios de correo académicos, para conseguir todos los centros educativos y sus dominios. Los usuarios que completan el proceso de *onboarding* podrán ver todas las asignaturas de su grado separadas por año y ver los apuntes que otros usuarios han compartido. Además podrán buscar globalmente entre todos los apuntes de la plataforma gracias a la utilización de *Full Text Search* permitiendo que la búsqueda sea instantánea aun con millones de apuntes.

La plataforma está construida siguiendo una arquitectura cliente-servidor: un backend desarrollado en Go, elegido por su rendimiento y la solidez de su ecosistema para la construcción de APIs, un frontend en SvelteKit que proporciona una interfaz

limpia y responsiva y PostgreSQL como base de datos lo que facilita ciertas funcionalidades.

Palabras clave

Aplicación Web, Plataforma Colaborativa, Recursos Académicos, Go, SvelteKit, PostgreSQL, OAuth2, Docker.

Abstract

Nemsys, a collaborative web platform for sharing and accessing academic resources

Until relatively recently, there were no well-integrated platforms in Spain where students could share notes, exams or other academic resources; they relied on in-person exchanges or messaging groups with no real persistence. In 2015 Wuolah was launched, a platform that did integrate with educational institutions and allowed users to upload notes linked specifically to a subject within a degree programme at their university. However, this platform suffers from significant performance limitations and an advertising-heavy monetisation model that degrades the user experience.

This Bachelor's Thesis describes the design and implementation of an alternative web application that allows students to share, search and download notes without interference, with an emphasis on speed and lightness from the outset. The platform is designed to integrate with subjects, degree programmes and educational institutions both in Spain and abroad. Due to time constraints and the tedious nature of the process rather than technical difficulty, full integration has only been completed for the UCM, although the architecture is ready for expansion to other institutions: only a *web crawler* adapted to each institution's website would be needed.

To achieve this, a combination of *web scraping* and the development of a script that uses JetBrains SWOT, a list of academic email domains, was used to obtain all educational institutions and their domains. Users who complete the onboarding process can see all the subjects in their degree organised by year and access notes shared by other students. They can also search globally across all notes on the platform thanks to *Full Text Search*, allowing searches to remain instant even with millions of resources.

The platform is built following a client-server architecture: a backend developed in Go, chosen for its performance and the robustness of its ecosystem for building APIs, a frontend in SvelteKit that provides a clean and responsive interface, and PostgreSQL as the database which facilitates certain functionalities.

Keywords

Web Application, Collaborative Platform, Academic Resources, Go, SvelteKit, PostgreSQL, OAuth2, Docker.

Índice general

Resumen	3
Nemsys, una plataforma web colaborativa para compartir y acceder a recursos académicos	3
Palabras clave	4
Abstract	6
Nemsys, a collaborative web platform for sharing and accessing academic resources	6
Keywords	7
1 Introducción	1
1.1 Motivación	1
1.2 Objetivos	3
1.3 Plan de trabajo	4
1.4 Estructura del documento	5
1 Introduction	6
1.1 Motivation	6
1.2 Objectives	8
1.3 Work Plan	9
1.4 Document Structure	9
2 Estado de la Cuestión	10
2.1 Plataformas existentes	10
2.1.1 Wuolah	10
2.1.2 Studocu	11
2.1.3 Docsity	12
2.1.4 Conclusiones	13
3 Descripción del Trabajo	14
3.1 Tecnologías evaluadas	14
3.1.1 Backend	14
3.1.2 Frontend	17
3.1.3 Base de datos	19
3.1.4 Almacenamiento	20
3.1.5 Despliegue	21
3.2 Arquitectura del sistema	21
3.2.1 Frontend	22
3.2.2 Backend	23

3.2.3	Autenticación	24
3.2.4	Despliegue	25
3.3	Diseño de la interfaz de usuario	28
3.3.1	Evolución del diseño	28
3.3.2	Sistema de diseño	29
3.3.3	Tipografía	30
3.3.4	Paleta de color	30
3.4	Diseño de la base de datos	31
3.4.1	Entidades principales	32
3.4.2	Evolución mediante migraciones	33
3.4.3	Búsqueda de texto completo	34
3.4.4	Índices adicionales	35
3.5	Implementación del backend	35
3.6	Implementación del frontend	36
3.7	Pruebas y validación	36
3.7.1	Tests unitarios del backend	36
3.7.2	Tests unitarios del frontend	38
3.7.3	Tests end to end con Playwright	39
3.7.4	Validación de rendimiento	41
4	Conclusiones y Trabajo Futuro	43
4.1	Conclusiones	43
4.2	Objetivos alcanzados	43
4.3	Trabajo futuro	43
	Bibliografía	44

Capítulo 1 Introducción

Resumen: En este capítulo se presenta la motivación del proyecto, los objetivos que se persiguen y la estructura del documento.

La calidad de un sistema no está determinada por el poder de sus componentes individuales, sino por cómo se eligen y se ensamblan para satisfacer las necesidades.

— Grady Booch

1.1 Motivación

Como estudiante de ingeniería de software utilizo una gran cantidad de software distinto en mi día a día; algunos funcionan mejor y otros resultan frustrantes. Es precisamente uno de los segundos lo que me motivó a realizar este Trabajo de Fin de Grado.

Antes de Wuolah, las alternativas se limitaban a plataformas genéricas como Docsity o StudyLib, que no se habían orientado al mercado español y cuyo contenido era muy escaso o prácticamente inexistente según la plataforma, y que tampoco se integraban con centros educativos concretos, o a soluciones informales como grupos de mensajería y *drives* compartidos.

Wuolah, la plataforma para compartir apuntes mejor integrada y más conocida, solucionó un problema real al ver la necesidad de los estudiantes de tener un lugar donde compartir y encontrar apuntes relevantes para sus asignaturas concretas, de su grado o estudio, en su centro educativo. A pesar de su popularidad el modelo actual que sigue presenta una serie de problemas que perjudican tanto la experiencia de usuario como la calidad del intercambio de información.

Su principal problema es el rendimiento. Al realizar un análisis de la plataforma utilizando Lighthouse, una herramienta de código abierto de Google que audita el rendimiento, la accesibilidad, mejores prácticas y el SEO de cualquier página web (Figura 1), se observan unos resultados bastante concluyentes.

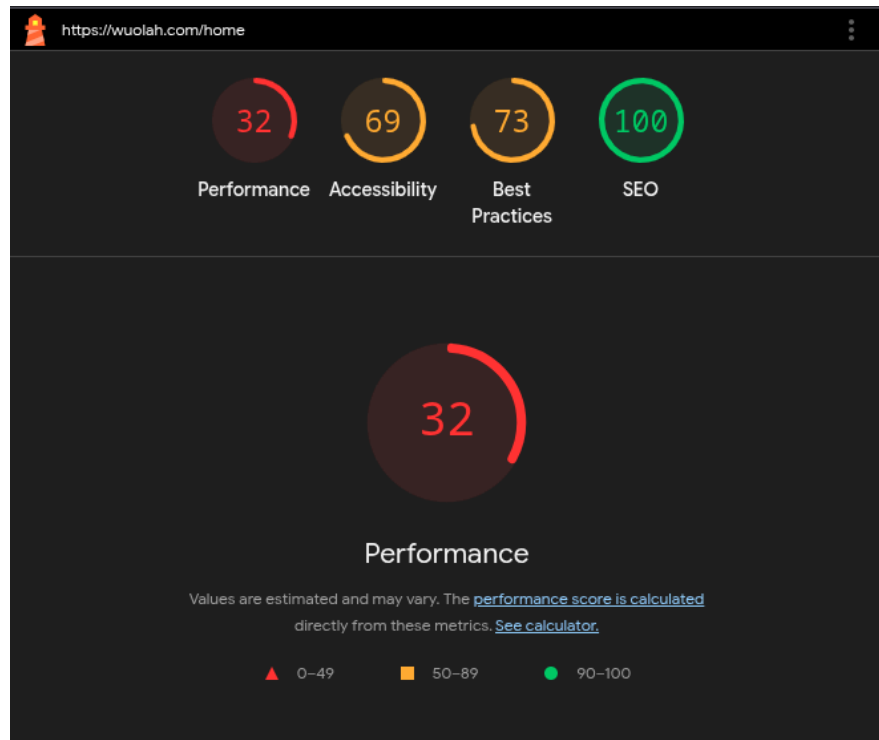


Figura 1: Análisis del rendimiento de wuolah.com con Google Lighthouse.

La plataforma obtiene una puntuación de rendimiento de 32 sobre 100, un resultado que muestra un gran problema de optimización y explica la lentitud que se siente al usarla. Esta baja puntuación se puede explicar por el uso de una arquitectura pesada en el lado del cliente y la gran cantidad de anuncios que se cargan en este. Esto empeora significativamente la experiencia del usuario y demuestra la necesidad de un enfoque de diseño más eficiente.

Además, su modelo de negocio basado en publicidad intensiva también perjudica la experiencia. Si bien es comprensible que la plataforma necesite monetizarse, la implementación actual penaliza en exceso al usuario gratuito con largos tiempos de espera para descargar cada recurso, a menos que se pague una suscripción mensual.

Por otro lado los micropagos por descarga que ofrece Wuolah presentan otro problema fundamental. Cuando se introduce una recompensa económica, especialmente cuando es mínima, se corre el riesgo de transformar la motivación de los usuarios; el deseo genuino de ayudar es reemplazado por un interés transaccional de bajo valor. Este enfoque ha sido demostrado como algo que puede llegar a ser contraproducente. La investigación sobre la motivación ha demostrado que las recompensas externas pueden disminuir el interés propio por realizar una tarea, un efecto conocido como «Desplazamiento Motivacional» [1], [2].

Además, este modelo de incentivos tiene otra consecuencia negativa para las carreras técnicas. Al limitar las recompensas a los archivos PDF, ya que en estos es en los que se puede incrustar anuncios, se desincentiva a que los estudiantes compartan información en otros formatos más adecuados para ciertas áreas como puede ser en el

caso de código, empobreciendo así la variedad y utilidad de los recursos disponibles en la plataforma.

Por último, cabe mencionar también cómo la interfaz actual refleja un problema común en plataformas que reciben un porcentaje significativo de uso desde dispositivos móviles: para facilitar la responsividad, la experiencia en escritorio se ha visto degradada. Como se puede observar en la Figura 2, con una resolución estándar de portátil de 1920x1080 al 100% de zoom, apenas la mitad de la pantalla se dedica al contenido, dejando las barras laterales sobrantes para anuncios. De esa mitad usada, algo más de un tercio está ocupado por una barra lateral con información de poco valor, dejando apenas una fracción del ancho total de la pantalla para lo que el usuario realmente quiere ver.

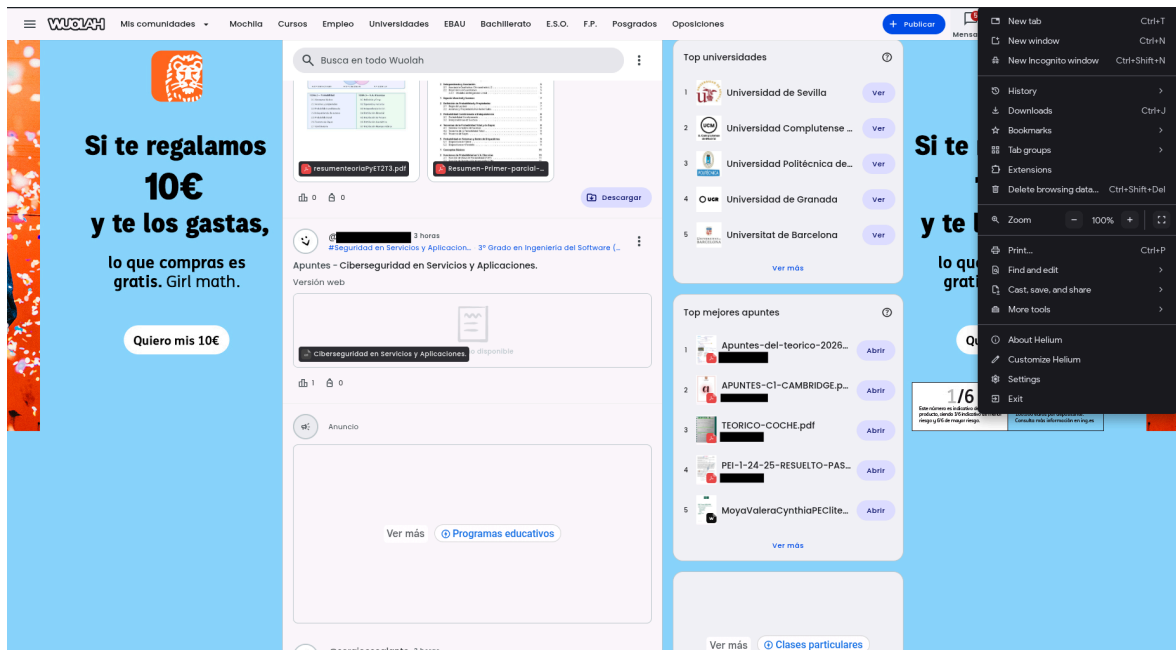


Figura 2: Captura de pantalla de wuolah.com mostrando la interfaz actual.

Este trabajo busca solucionar todos estos problemas y tiene como objetivo el diseño y desarrollo de un prototipo de plataforma alternativa. La solución propuesta mejorará el rendimiento usando una arquitectura ligera y eficiente con un *backend* en Go y un *frontend* con SvelteKit. Para reducir los costes de mantenimiento se usarán servidores privados virtuales (VPS) en vez de plataformas en la nube como AWS. Y se eliminará la publicidad invasiva y los incentivos monetarios para fomentar la colaboración genuina y la compartición de recursos en múltiples formatos.

1.2 Objetivos

El objetivo principal de este proyecto es crear y desplegar un prototipo de una aplicación para compartir recursos académicos como apuntes y ejercicios resueltos entre otros. Para realizarlo se han definido los siguientes objetivos:

1. Diseñar e implementar una API REST en Go que gestione usuarios, recursos y asignaturas.
2. Desarrollar una interfaz web con SvelteKit que ofrezca una experiencia rápida y sin publicidad.
3. Integrar autenticación con Google OAuth2 como método único de acceso, detectando automáticamente el centro educativo del usuario a partir de su dominio de correo.
4. Obtener automáticamente centros educativos y sus asignaturas mediante *web scraping* y JetBrains SWOT.
5. Utilizar almacenamiento de objetos compatible con S3 para gestionar los archivos subidos por los usuarios.
6. Implementar búsqueda global mediante *Full Text Search* de PostgreSQL.
7. Desplegar la plataforma de forma portable usando Docker Compose en un VPS.

1.3 Plan de trabajo

El desarrollo del proyecto se ha llevado a cabo siguiendo un enfoque iterativo e incremental con una metodología *API-first*: para cada funcionalidad se implementaba primero el *endpoint* en el *backend* y después la interfaz correspondiente. Para la gestión de tareas se utilizó un tablero Kanban en Taiga. Las fases principales del desarrollo fueron las siguientes:

1. **Prototipado inicial (septiembre - octubre 2025).** Elección de tecnologías, implementación de la autenticación con Google OAuth2, primeras iteraciones del diseño de la interfaz y configuración inicial de la base de datos y el flujo de *onboarding*.
2. **Diseño de la arquitectura e implementación de las funcionalidades principales (enero - febrero 2026).** Definición de la API REST, sustituyendo un enfoque inicial con GraphQL que resultó innecesario para el proyecto. Desarrollo del *scraper* de la UCM, implementación de la subida, descarga y previsualización de recursos, soporte para múltiples archivos por recurso y diseño del sistema de vistas.
3. **Funcionalidades avanzadas y refinamiento (marzo 2026).** Perfiles de usuario, búsqueda global con *Full Text Search*, integración de universidades mediante JetBrains SWOT y *web scraping*, y mejoras generales de la interfaz.
4. **Pruebas, despliegue y documentación (abril 2026).** Tests unitarios del backend, tests unitarios y *end to end* del *frontend* con Playwright, configuración de Docker y Docker Compose, despliegue en VPS y redacción de la memoria.

1.4 Estructura del documento

Este documento está estructurado de la siguiente manera:

- **Capítulo 2. Estado de la Cuestión:** Se analizan las plataformas existentes para compartir recursos académicos.
- **Capítulo 3. Descripción del Trabajo:** Se justifican las decisiones tecnológicas y se describe en detalle la arquitectura, el diseño y la implementación de la plataforma.
- **Capítulo 4. Conclusiones y Trabajo Futuro:** Se presentan las conclusiones y las posibles líneas de mejora.

Capítulo 1 Introduction

Abstract: This chapter presents the motivation behind the project, the objectives pursued and the structure of the document.

The quality of a system is not determined by the power of its individual components, but by how they are chosen and assembled to meet the needs.

— Grady Booch

1.1 Motivation

As a software engineering student I use a large amount of different software in my day to day life; some work well and some are frustrating. It is precisely one of the latter that motivated me to undertake this Bachelor's Thesis.

Before Wuolah, the alternatives were limited to generic platforms such as Docsity or StudyLib, which had not been targeted at the Spanish market and therefore had very little or practically no content depending on the platform, and which also did not integrate with specific educational institutions, or to informal solutions such as messaging groups and shared drives.

Wuolah, the best known and most integrated platform for sharing notes, solved a real problem by recognising the need for students to have a place to share and find relevant notes for their specific subjects, in their degree programme and educational institution. Despite its popularity, its current model presents a series of problems that harm both the user experience and the quality of information exchange.

Its main problem is performance. By analysing the platform using Lighthouse, an open source Google tool that audits the performance, accessibility, best practices and SEO of any website (Figure 3), some fairly conclusive results are obtained.

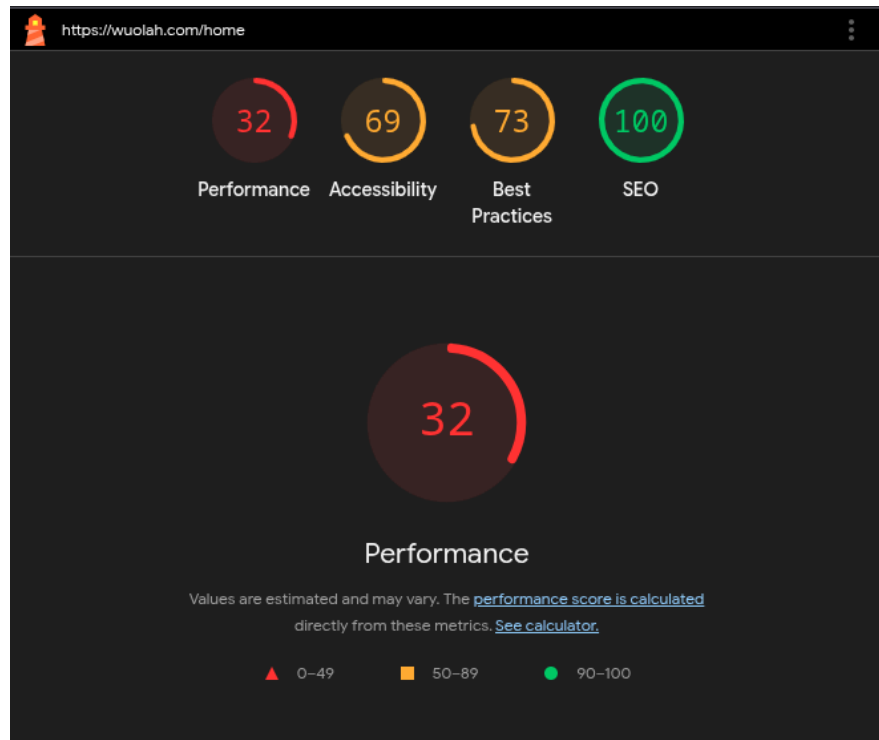


Figure 3: Performance analysis of wuolah.com with Google Lighthouse.

The platform achieves a performance score of 32 out of 100, a result that reveals a major optimisation problem and explains the slowness felt when using it. This low score can be explained by the use of a heavy client-side architecture and the large number of advertisements loaded on it. This significantly worsens the user experience and demonstrates the need for a more efficient design approach.

Furthermore, its business model based on intensive advertising also harms the experience. While it is understandable that the platform needs to monetise itself, the current implementation excessively penalises the free user with long waiting times to download each resource, unless a monthly subscription is paid.

On the other hand, the micropayments per download offered by Wuolah present another fundamental problem. When an economic reward is introduced, especially when it is minimal, there is a risk of transforming users' motivation; the genuine desire to help is replaced by a transactional interest of low value. This approach has been shown to be potentially counterproductive. Research on motivation has demonstrated that external rewards can diminish intrinsic interest in performing a task, an effect known as "Motivational Displacement" [1], [2].

Furthermore, this incentive model has another negative consequence for technical degrees. By limiting rewards to PDF files, since these are the ones in which advertisements can be embedded, it discourages students from sharing information in other formats more suitable for certain areas such as code, thus impoverishing the variety and usefulness of the resources available on the platform.

Finally, it is also worth mentioning how the current interface reflects a common problem in platforms that receive a significant percentage of usage from mobile

devices: to facilitate responsiveness, the desktop experience has been degraded. As can be seen in Figure 4, with a standard laptop resolution of 1920×1080 at 100% zoom, barely half of the screen is devoted to content, leaving the remaining side bars for advertisements. Of that half, slightly more than a third is occupied by a sidebar with low value information, leaving barely a fraction of the total screen width for what the user actually wants to see.

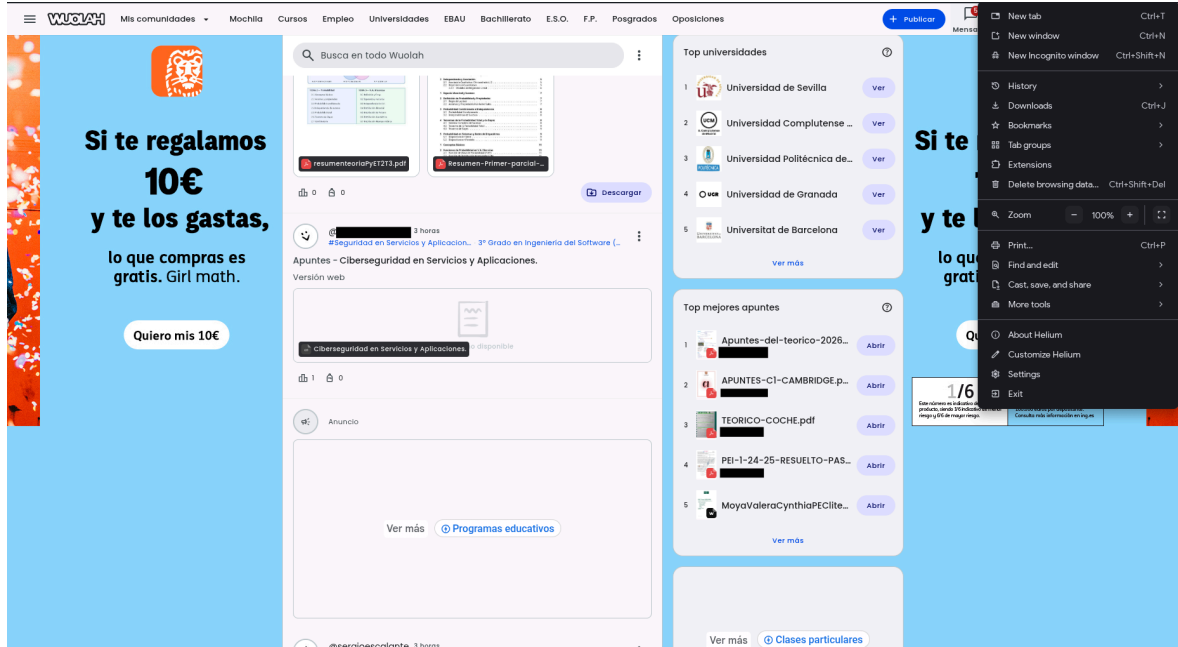


Figure 4: Screenshot of wuolah.com showing the current interface.

This work seeks to solve all these problems and aims to design and develop a prototype of an alternative platform. The proposed solution improves performance using a lightweight and efficient architecture with a Go backend and a SvelteKit frontend. To reduce maintenance costs, virtual private servers (VPS) are used instead of cloud platforms such as AWS. Intrusive advertising and monetary incentives are eliminated to foster genuine collaboration and the sharing of resources in multiple formats.

1.2 Objectives

The main objective of this project is to create and deploy a prototype of an application for sharing academic resources such as notes and solved exercises, among others. To achieve this, the following objectives have been defined:

1. Design and implement a REST API in Go that manages users, resources and subjects.
2. Develop a web interface with SvelteKit that offers a fast, advertisement free experience.

3. Integrate authentication with Google OAuth2 as the sole access method, automatically detecting the user's educational institution from their email domain.
4. Automatically retrieve educational institutions and their subjects through web scraping and JetBrains SWOT.
5. Use S3-compatible object storage to manage files uploaded by users.
6. Implement global search using PostgreSQL Full Text Search.
7. Deploy the platform in a portable manner using Docker Compose on a VPS.

1.3 Work Plan

The development of the project was carried out following an iterative and incremental approach with an API-first methodology: for each feature, the backend endpoint was implemented first, followed by the corresponding interface. For task management, a Kanban board on Taiga was used. The main development phases were as follows:

1. **Initial prototyping (September – October 2025).** Technology selection, implementation of Google OAuth2 authentication, first iterations of the interface design and initial configuration of the database and the onboarding flow.
2. **Architecture design and implementation of core features (January – February 2026).** Definition of the REST API, replacing an initial GraphQL approach that proved unnecessary for the project. Development of the UCM scraper, implementation of resource uploading, downloading and preview, support for multiple files per resource, and design of the view system.
3. **Advanced features and refinement (March 2026).** User profiles, global search with Full Text Search, university integration via JetBrains SWOT and web scraping, and general interface improvements.
4. **Testing, deployment and documentation (April 2026).** Backend unit tests, frontend unit and end to end tests with Playwright, Docker and Docker Compose configuration, VPS deployment and writing of the thesis report.

1.4 Document Structure

This document is structured as follows:

- **Chapter 2. State of the Art:** Existing platforms for sharing academic resources are analysed.
- **Chapter 3. Work Description:** The technological decisions are justified and the architecture, design and implementation of the platform are described in detail.
- **Chapter 4. Conclusions and Future Work:** Conclusions and possible lines of improvement are presented.

Capítulo 2 Estado de la Cuestión

Resumen: Este capítulo analiza el estado del arte de las plataformas colaborativas para compartir recursos académicos.

2.1 Plataformas existentes

A continuación se analizan las tres plataformas más relevantes para compartir recursos académicos, evaluando su integración con centros educativos, su modelo de monetización y la experiencia de usuario que ofrecen.

2.1.1 Wuolah

Wuolah es una plataforma española que ha sabido integrar las asignaturas, cursos y centros educativos para que sus usuarios puedan compartir y acceder a apuntes y exámenes verdaderamente relevantes para ellos. No se centra únicamente en estudios universitarios, también ofrece soporte para E.S.O, Bachillerato, EBAU, Ciclos y Oposiciones. Además, es la única de las plataformas analizadas que ofrece una compensación monetaria a los usuarios que publican sus apuntes, un aspecto cuyas implicaciones ya se han discutido en la motivación de este trabajo.

El proceso de registro es sencillo para lo que cubre la plataforma ya que intenta abarcar mucho más que solo estudios universitarios. Se puede iniciar sesión con correo y contraseña o mediante OAuth2, tras lo cual se selecciona el país, el tipo de estudio, el estudio concreto y el centro educativo. Sin embargo, el último paso requiere aceptar su política de privacidad, cuya presentación incluye patrones oscuros como lenguaje emocional y apelaciones a la urgencia del estudiante (Figura 5). Además, junto a la política se solicita consentimiento para recibir publicidad de empresas de sectores ajenos a la educación.

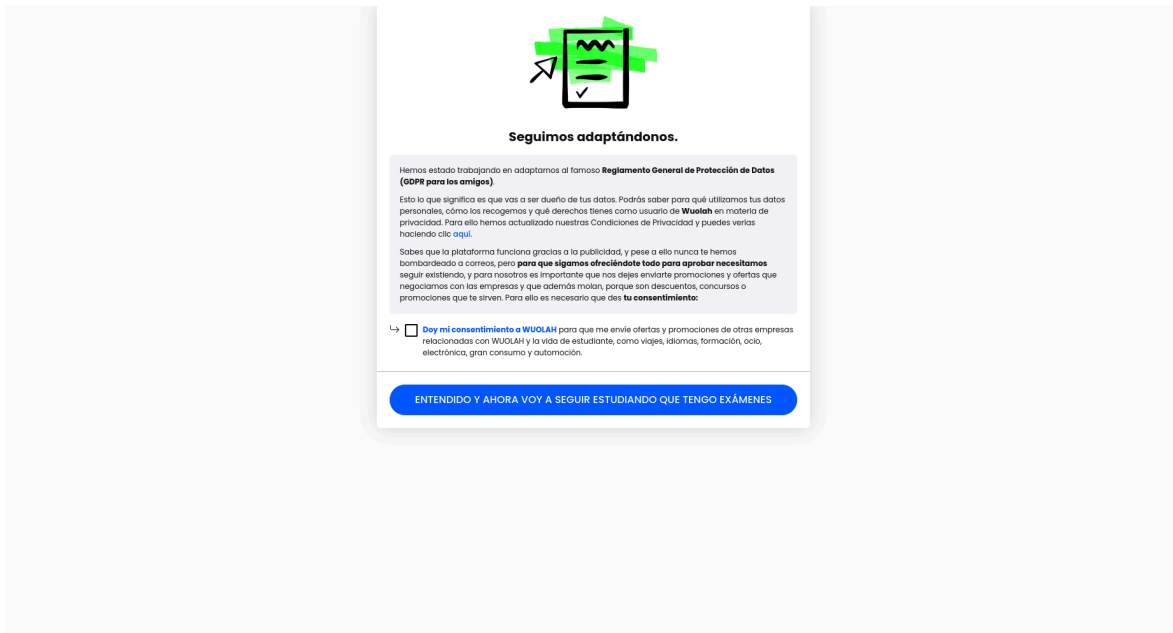


Figura 5: Pantalla de aceptación de la política de privacidad de Wuolah.

Esto va acompañado de los problemas anteriormente mencionados como el exceso de publicidad que interfiere con el uso de la plataforma, tiempos de espera de hasta medio minuto para que un usuario gratuito pueda descargar un apunte, y una interfaz de escritorio poco eficiente para facilitar la responsividad en móvil. A esto se suma una barra de navegación con demasiada información: si un usuario ya ha indicado que cursa un grado universitario, de poco le sirve tener acceso directo a los apartados de E.S.O, Bachillerato o EBAU (Figura 6).



Figura 6: Barra de navegación de Wuolah.

En conclusión, Wuolah es la plataforma que mejor ha resuelto el flujo de usuario para compartir recursos académicos y, con diferencia, la que mayor volumen de contenido tiene para universidades españolas. Sin embargo, sus defectos han ido acumulándose con el tiempo a medida que la plataforma ha crecido en funcionalidades y ha intensificado su modelo de monetización.

2.1.2 Studocu

Studocu es una plataforma fundada en 2013 en los Países Bajos que ha crecido hasta convertirse en la mayor plataforma de apuntes a nivel global. También ofrece integración con centros educativos, aunque algo más superficial que la de Wuolah, permite seleccionar universidad y asignaturas, pero no grado, por lo que asignaturas de mismo nombre compartidas entre grados se agrupan aunque puedan tener un temario diferente.

Dejando ese detalle de lado, como se puede ver en la Figura 7, la interfaz de escritorio es excelente, aprovecha bien el ancho de la pantalla y tiene una buena

densidad de información. Sin embargo, al no ser tan popular en España, muchas asignaturas de grados como Ingeniería de Software no aparecen, y las que sí lo hacen tienen varias veces menos contenido que en Wuolah.

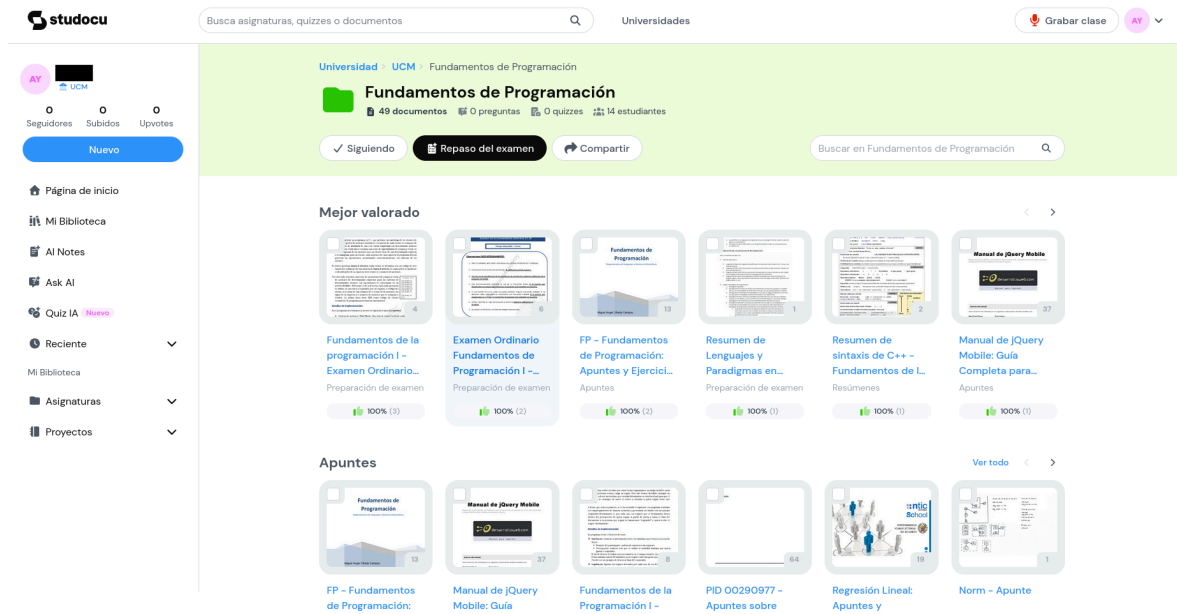


Figura 7: Captura de pantalla de Studocu.

Por último, aunque lejos del alcance de este TFG, cabe destacar que Studocu cuenta con una gran implementación de inteligencia artificial que recopila automáticamente la información de la asignatura y ayuda al estudiante a repasar con ese contexto. Wuolah también ofrece una funcionalidad similar, aunque todavía en fase beta y más limitada, siendo un *chatbot* al que se le pueden adjuntar apuntes concretos.

2.1.3 Docsity

Docsity es una plataforma italiana lanzada en 2010 que se abrió al mercado internacional en 2012. Ofrece una integración con centros educativos que incluye universidad, área de estudio, asignaturas y carrera, aunque al igual que Studocu no hay segregación de asignaturas por grado, por lo que la selección de carrera no tiene un efecto real en el contenido que se muestra.

La plataforma utiliza un sistema de puntos. Para incentivar las contribuciones los usuarios obtienen puntos al subir documentos o responder preguntas, y los gastan al descargar contenido de otros. Si bien evita la publicidad tan agresiva de Wuolah, este sistema de puntos puede resultar frustrante cuando un usuario quiere acceder a un recurso puntual y no tiene saldo suficiente.

En cuanto a presencia en España, Docsity cuenta con un volumen notable de contenido para algunas universidades como la UCM, aunque su distribución es desigual, carreras técnicas como Ingeniería de Software apenas tienen recursos disponibles, concentrándose la mayoría del contenido en áreas como periodismo, derecho o historia.

La interfaz (Figura 8) es limpia pero algo más confusa que las de Wuolah o Studocu ya que dentro de una asignatura solo los documentos populares muestran una miniatura, mientras que el resto se presenta en una lista sin previsualización, y la disposición general resulta menos intuitiva.

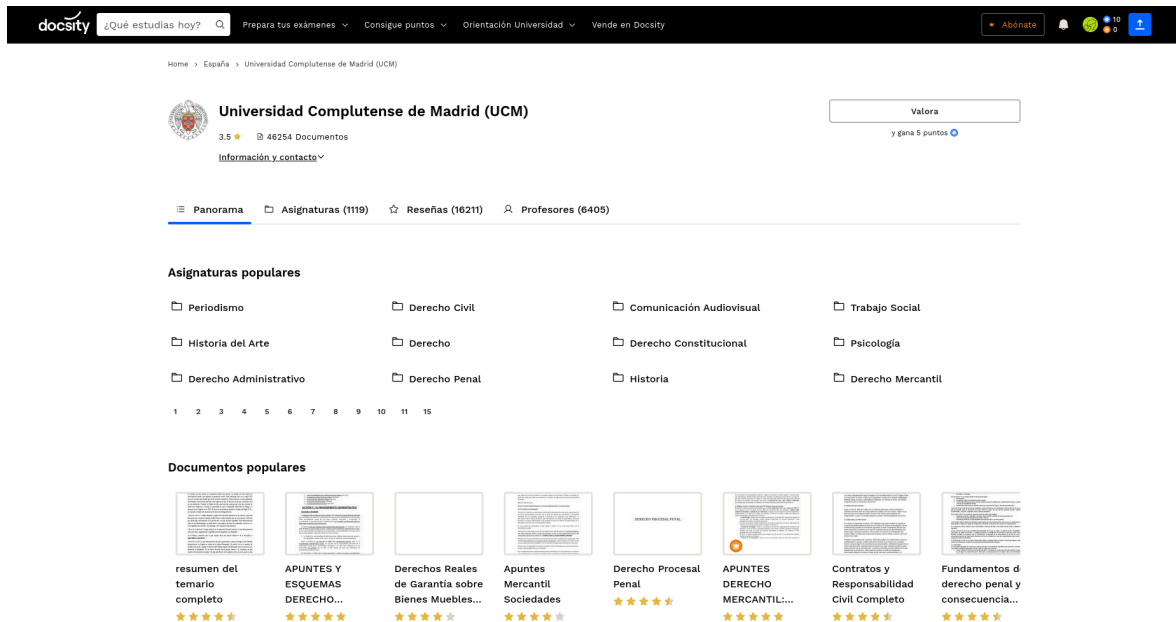


Figura 8: Captura de pantalla de Docsity para la UCM.

2.1.4 Conclusiones

Las tres plataformas analizadas comparten un objetivo común pero presentan carencias distintas. Wuolah ofrece la mejor integración con centros educativos españoles pero a costa del rendimiento y la experiencia de usuario. Studocu destaca por su interfaz y su alcance global, aunque su integración es más superficial y su contenido en España es limitado. Docsity, por su parte, tiene un modelo de incentivos menos intrusivo pero una organización del contenido poco precisa. Ninguna de las tres combina una integración completa con centros educativos, un buen rendimiento y una experiencia de usuario libre de publicidad o restricciones artificiales, lo que justifica la propuesta desarrollada en este trabajo.

Capítulo 3 Descripción del Trabajo

Resumen: En este capítulo se justifican las decisiones tecnológicas y se describe en detalle el desarrollo de la plataforma Nemsy.

3.1 Tecnologías evaluadas

En esta sección se justifican las principales decisiones tecnológicas del proyecto, comparando las opciones consideradas para cada componente de la arquitectura.

3.1.1 Backend

Para el backend se evaluaron tres alternativas con las que ya tenía experiencia: Node.js con Express, Spring Boot (Java) y Go con Chi, siendo esta última con la que más cómodo me sentía. Dos de ellas figuran entre los frameworks web más utilizados según la encuesta anual de Stack Overflow a desarrolladores [3], como se puede observar en la Figura 9.

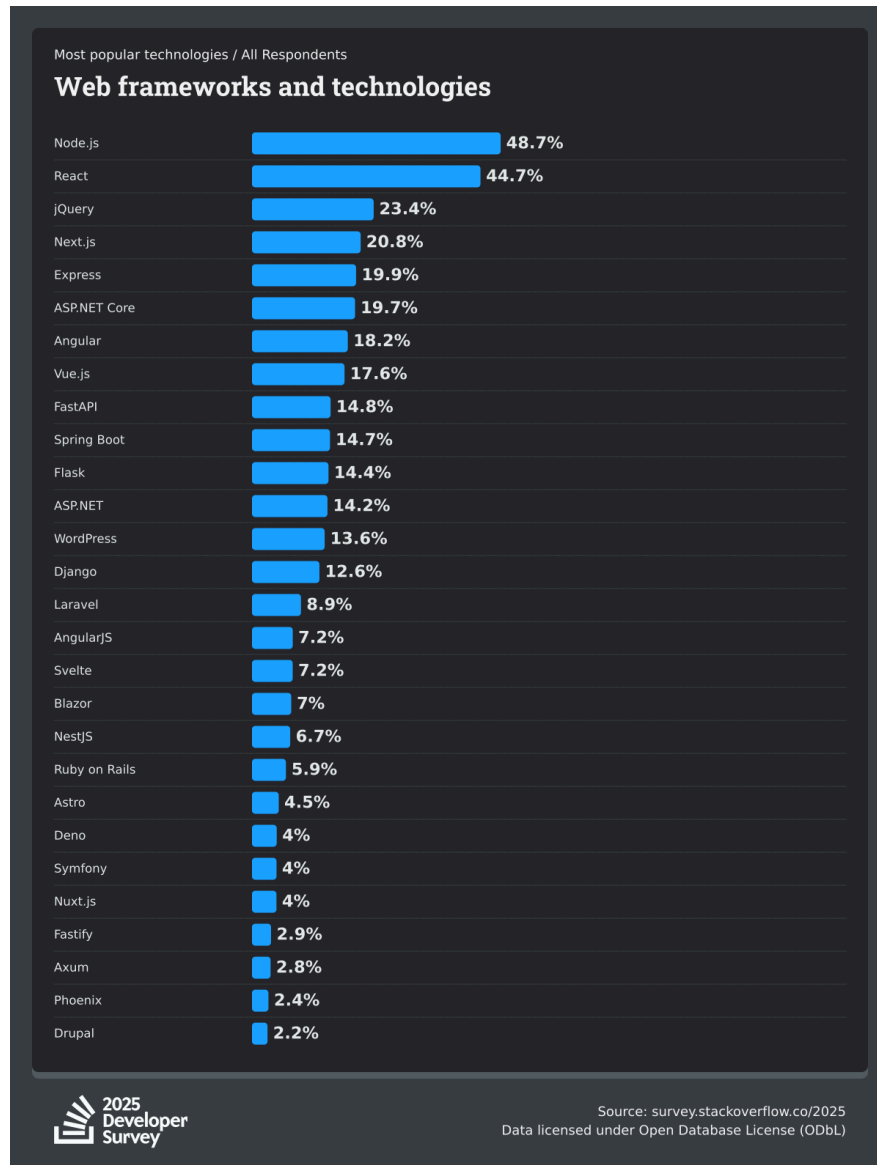


Figura 9: Frameworks y tecnologías web más populares según la encuesta de Stack Overflow 2025 [3].

3.1.1.1 Node.js (Express)

Express es un framework web minimalista escrito sobre Node.js. Como refleja la Figura 9, se sitúa entre los frameworks web más utilizados a nivel global, lo que se traduce en un ecosistema muy maduro y una gran cantidad de recursos disponibles. Además, es el framework utilizado en la asignatura de Aplicaciones Web del grado, por lo que ya se contaba con experiencia previa en su uso.

En cuanto a la experiencia de desarrollo, Express es excelente, es directo y sencillo, y su principal punto fuerte es la velocidad con la que se puede construir un servicio. Sin embargo, el objetivo de este proyecto no es maximizar la velocidad de desarrollo, sino ofrecer una plataforma que se sienta rápida para el usuario final, y Node.js, al ser interpretado y basarse en un único hilo de ejecución, no es la opción más adecuada para ello. Es por ello que fue descartada.

3.1.1.2 Spring Boot

Spring Boot es el framework más popular del ecosistema Java y también aparece en la Figura 9 con una presencia notable. Proporciona un entorno muy completo con inyección de dependencias, ORM integrado y una amplia cobertura para aplicaciones empresariales. Este es el framework utilizado en la asignatura de Ingeniería Web del grado, por lo que también se contaba con experiencia previa en su uso.

Su experiencia de desarrollo también es destacable una vez superado el boilerplate inicial, aunque con un matiz. La filosofía del ecosistema Java se apoya en múltiples capas de abstracción, lo cual puede dificultar la depuración cuando algo falla. Además, para un proyecto de este tamaño, contar con ORM e inyección de dependencias resulta más una carga que una ventaja. A esto se suma el elevado consumo de memoria de la JVM, un factor relevante al desplegar en un VPS con recursos limitados. Por todo ello, también fue descartada.

3.1.1.3 Go (Chi)

Go es un lenguaje de programación creado por Google, compilado y con un fuerte enfoque en servicios web y cloud [4]. Su librería estándar incluye el módulo `net/http`, lo suficientemente sólido como para construir un servidor HTTP sin dependencias externas. Chi es una extensión ligera sobre este módulo que añade utilidades como el enrutamiento con parámetros y middlewares encadenables [5]. Además, Go cuenta con un sistema de testing integrado en el propio lenguaje y un tooling moderno que cubre desde el formateo de código hasta la gestión de dependencias.

Frente a la filosofía de Java, donde las abstracciones ocultan los detalles de implementación, el ecosistema de Go apuesta por la generación de código y la transparencia. Herramientas como `sqlc` generan código Go a partir de consultas SQL escritas a mano, de forma que el desarrollador mantiene el control total sobre las queries sin depender de un ORM. Esto resulta especialmente relevante cuando se quieren aprovechar funcionalidades específicas del motor de base de datos elegido, algo que un ORM tiende a dificultar y cuya importancia se detallará en la sección de base de datos.

3.1.1.4 Comparativa

Criterio	Node.js 	Spring Boot 	Go 
Rendimiento	Medio (V8, single-thread)	Alto (JVM, JIT)	Alto (compilado, nativo)
Concurrencia	Event loop, un hilo	Threads del SO	Goroutines (ligeras)
Consumo de recursos	Medio	Alto (JVM)	Bajo (binario único)
Ecosistema HTTP	Muy amplio (Express)	Muy amplio (Spring MVC)	Sólido (net/http, Chi)
Despliegue	Requiere Node + node_modules	Requiere JVM + JAR	Binario único
Curva de aprendizaje	Ya conocido	Ya conocido	Ya conocido

Tabla 1: Comparativa de tecnologías para el backend.

Como se resume en la Tabla 1, Go destaca frente a las otras dos alternativas en consumo de recursos y despliegue, al compilar a un binario único sin dependencias externas. Además, su modelo de concurrencia basado en goroutines permite manejar operaciones simultáneas como descargas de archivos de forma nativa, sin configuración adicional. Dado que el foco de este TFG está muy ligado a la velocidad y el bajo consumo de recursos de la plataforma, Go resultó la opción más adecuada.

3.1.2 Frontend

Para el frontend se consideraron tres enfoques con filosofías distintas: un framework completo con React (Next.js), un framework ligero con Svelte (SvelteKit), y el enfoque clásico de servidor con templating y Bootstrap, utilizado en varias asignaturas del grado como Aplicaciones Web (EJS) e Ingeniería Web (Thymeleaf).

3.1.2.1 Servidor con templating + Bootstrap

El enfoque más directo y el más familiar por las asignaturas del grado. Con motores como EJS o Thymeleaf, el HTML se renderiza en el servidor y se envía completo al navegador, mientras que Bootstrap proporciona estilos predefinidos y componentes básicos. Es sencillo de entender y desplegar, pero presenta limitaciones claras para una aplicación interactiva: cada acción del usuario requiere una recarga completa de la página, la experiencia resulta lenta y poco fluida, y la personalización visual está limitada por los estilos de Bootstrap. Para una plataforma que aspira a competir en experiencia de usuario con aplicaciones modernas, este enfoque resulta insuficiente.

3.1.2.2 Next.js (React)

Next.js es el meta framework más popular del ecosistema React y uno de los más utilizados en la industria, como refleja la Figura 9. Ofrece renderizado del lado del servidor (SSR), generación estática, enrutamiento basado en ficheros y un ecosistema de componentes y librerías enorme. Sin embargo, React se basa en un virtual DOM que añade una capa de abstracción entre el código y el navegador [6], lo que resulta en un bundle más pesado. Además, no tenía experiencia previa con React, lo que habría añadido una curva de aprendizaje significativa al proyecto.

3.1.2.3 SvelteKit (Svelte)

SvelteKit es un meta-framework construido sobre Svelte, un compilador que transforma los componentes a JavaScript vanilla durante el build, eliminando la necesidad de un virtual DOM en tiempo de ejecución [7]. Esto resulta en bundles significativamente más pequeños y un mejor rendimiento en el navegador. Ya tenía experiencia con Svelte y SvelteKit, y además Svelte 5 introduce las runes, un sistema de reactividad más explícito y eficiente que el de versiones anteriores.

SvelteKit ofrece las mismas capacidades que Next.js (SSR, enrutamiento basado en ficheros, layouts anidados) pero con un enfoque más ligero. Al igual que con Go en el backend, la elección se alinea con el objetivo del proyecto de ofrecer una experiencia rápida y eficiente.

3.1.2.4 Comparativa

Criterio	Templating + Bootstrap 	Next.js (React) 	SvelteKit (Svelte) 
Rendimiento	Bajo (recarga completa)	Medio (virtual DOM)	Alto (compilado, sin virtual DOM)
Interactividad	Mínima (MPA)	Alta (SPA/SSR)	Alta (SPA/SSR)
Tamaño del bundle	Bajo (sin framework de JS)	Alto	Muy bajo
Personalización IU	Limitada (Bootstrap)	Total	Total
Ecosistema	Maduro pero limitado	Muy amplio	Creciente
Curva de aprendizaje	Ya conocido	Nuevo	Ya conocido

Tabla 2: Comparativa de tecnologías para el frontend.

Se eligió SvelteKit por su combinación de rendimiento, tamaño de bundle reducido y mi familiaridad con el framework. Al compilar a JavaScript vanilla, se alinea directamente con el objetivo del proyecto de ofrecer una plataforma ligera y rápida.

3.1.3 Base de datos

Los datos de Nemsy son inherentemente relacionales. Los usuarios, universidades, asignaturas, recursos y archivos están interconectados mediante claves foráneas. Por ello, la decisión entre modelos de datos se inclinó desde el inicio hacia un sistema relacional, pero se evaluaron igualmente las alternativas con las que se tenía experiencia.

3.1.3.1 MySQL

MySQL es el SGBD relacional de código abierto más extendido y el utilizado en asignaturas del grado como Modelado de Software y Aplicaciones Web. Es robusto, bien documentado y cuenta con un ecosistema enorme [8]. Sin embargo, su soporte de búsqueda de texto completo (`FULLTEXT`) es limitado, no permite asignar pesos a distintos campos, carece de un sistema de ranking comparable a `ts_rank`, y no dispone de un mecanismo nativo para ignorar acentos, algo imprescindible en una plataforma en español. Para conseguir una experiencia de búsqueda equivalente habría sido necesario recurrir a un servicio externo como Elasticsearch, añadiendo complejidad al despliegue.

3.1.3.2 MongoDB

MongoDB es una base de datos documental NoSQL que almacena los datos en formato BSON (similar a JSON). Es una opción popular para prototipos rápidos y esquemas que cambian con frecuencia [9], y fue uno de los SGBD utilizado en la asignatura de Ampliación de Bases de Datos del grado. Sin embargo, el modelo de datos de Nemsy se basa en relaciones claras entre entidades, algo que MongoDB no gestiona de forma nativa, las consultas que implican joins requieren múltiples peticiones o desnormalizar los datos, lo que complica el mantenimiento y compromete la consistencia. Además, `sqlc`, la herramienta de generación de código elegida para el backend, solo soporta bases de datos SQL.

3.1.3.3 PostgreSQL

PostgreSQL es un SGBD relacional de código abierto con una licencia permisiva (PostgreSQL License), con el que ya contaba con experiencia por proyectos personales. Su principal ventaja frente a MySQL para este proyecto es el sistema de búsqueda de texto completo integrado [10]. Mediante columnas de tipo `tsvector`, índices GIN y la función `ts_rank`, PostgreSQL permite construir búsquedas con pesos por campo y ranking de relevancia sin dependencias externas. Además, la extensión `unaccent` permite ignorar tildes en las consultas, algo fundamental para texto en español. Estas funcionalidades se integran directamente en el esquema mediante columnas generadas, de forma que el índice de búsqueda se actualiza automáticamente con cada inserción o modificación.

3.1.3.4 Comparativa

Criterio	MySQL 	MongoDB 	PostgreSQL 
Modelo de datos	Relacional	Documental (NoSQL)	Relacional
Licencia	GPL	SSPL	PostgreSQL (permisiva)
Full-text search	Básico (FULLTEXT)	Básico	Avanzado (tsvector, ts_rank)
Soporte de acentos	No nativo	No nativo	Sí (unaccent)
Soporte en sqlc	Sí	No	Sí (nativo)
Curva de aprendizaje	Ya conocido	Ya conocido	Ya conocido

Tabla 3: Comparativa de sistemas de gestión de bases de datos.

Como se resume en la Tabla 3, PostgreSQL fue la elección natural. Es el único que resuelve la búsqueda de texto en español de forma nativa, sin añadir servicios externos al despliegue, y su compatibilidad con sqlc encaja directamente con la arquitectura del backend.

3.1.4 Almacenamiento

El almacenamiento de archivos es una pieza central en una plataforma como Nemsy, donde los usuarios suben y descargan documentos de forma continua. Se evaluaron dos enfoques para gestionar estos archivos.

La opción más simple es almacenar los archivos directamente en el sistema de ficheros del VPS. Es inmediata de implementar y no introduce dependencias externas, pero presenta limitaciones importantes, el espacio en disco de un VPS es limitado y caro por gigabyte, y más importante, los ficheros quedan ligados a una sola máquina y cualquier migración implica mover manualmente todo el contenido.

El almacenamiento de objetos es el enfoque estándar en la industria para este tipo de casos. AWS S3 es el servicio de referencia y su API se ha convertido en estándar de facto, pero su modelo de precios incluye costes de egreso que pueden resultar en facturas impredecibles en una plataforma orientada a descargas. Hetzner Object Storage ofrece la misma API compatible con S3 y un modelo de precios más predecible, con un coste fijo mensual por TB almacenado que incluye tráfico sin cargos adicionales por egreso. Al estar ubicado en centros de datos europeos, facilita además el cumplimiento del RGPD, un aspecto relevante para una plataforma que almacena documentos de usuarios de la Unión Europea.

La integración se realiza a través de la librería `minio-go`, que implementa el protocolo S3 de forma genérica y permite conectar con cualquier proveedor compatible mediante variables de entorno, sin cambios en el código.

3.1.5 Despliegue

La plataforma se despliega en un VPS con recursos limitados, por lo que la solución de despliegue debía ser ligera, reproducible y fácil de mantener por una sola persona.

Una alternativa habría sido desplegar los servicios manualmente, instalando Go, Node.js y PostgreSQL directamente en el sistema operativo del servidor y gestionando los procesos con `systemd`. Es el enfoque más simple en términos de herramientas, pero también el más frágil, ya que el entorno del servidor se convierte en un estado implícito difícil de reproducir y actualizar una dependencia puede romper el resto.

Kubernetes es la solución estándar para orquestación de contenedores a escala, pero introduce una complejidad operativa desproporcionada para un proyecto de este tamaño. Requiere un clúster, recursos considerables y conocimientos específicos de administración, aspectos que no se justifican cuando la plataforma corre en una sola máquina, pero que sí serían valiosos si en el futuro fuese necesario escalar a múltiples máquinas.

Docker Compose permite definir todos los servicios de la aplicación (backend, frontend y base de datos) en un único fichero `docker-compose.yml`, levantarlos con un solo comando y garantizar que el entorno es idéntico en local y en producción. Cada servicio corre en su propio contenedor aislado, las variables de entorno se gestionan mediante un fichero `.env`, y el estado persistente de PostgreSQL se mantiene en un volumen nombrado. Es la opción que mejor equilibra reproducibilidad y simplicidad en el contexto de este proyecto.

3.2 Arquitectura del sistema

Nemsys sigue una arquitectura cliente-servidor de tres capas. El frontend es una *Single Page Application* (SPA) desarrollada con SvelteKit que se comunica con el backend exclusivamente a través de una API REST. El backend, implementado en Go con el router Chi, actúa como única puerta de entrada al sistema gestionando la autenticación, la lógica de negocio y el acceso a los dos sistemas de persistencia, PostgreSQL y un almacenamiento de objetos compatible con S3. El conjunto se despliega mediante Docker Compose en un VPS, de forma que el entorno de producción es reproducible e idéntico al de desarrollo local.

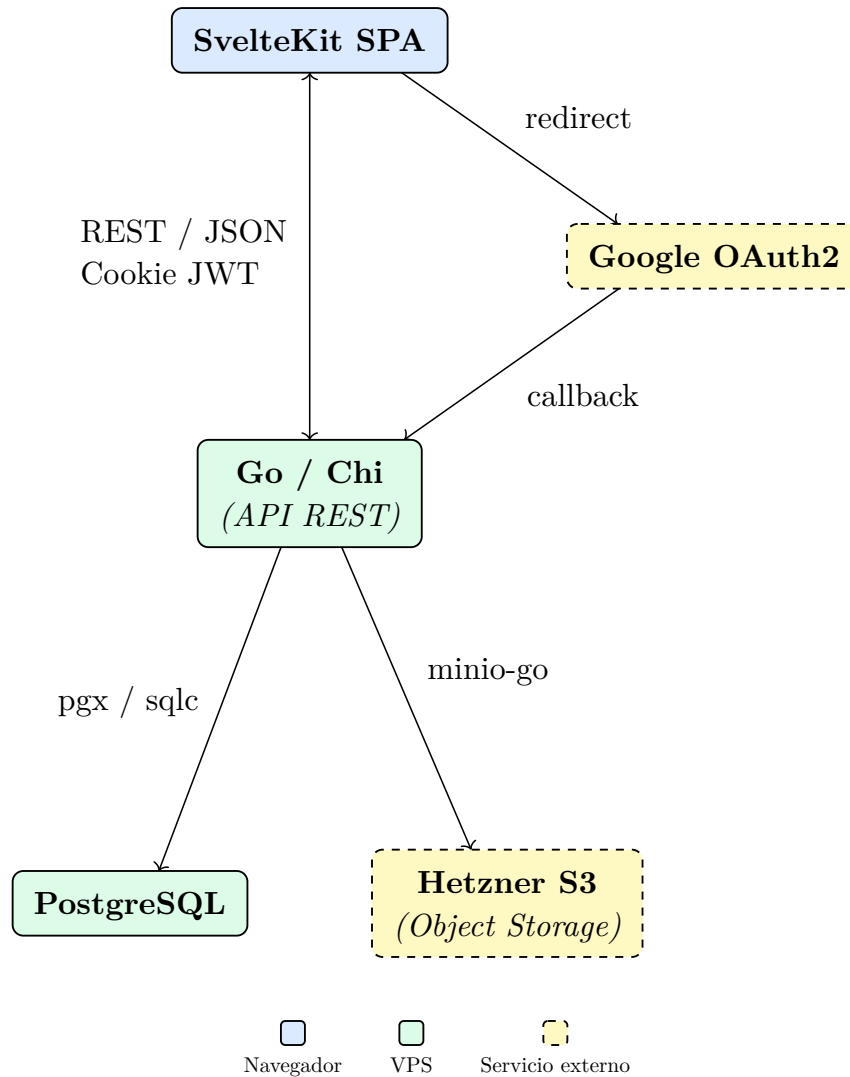


Figura 10: Diagrama de componentes de la arquitectura de Nemsy.

3.2.1 Frontend

El frontend está construido con SvelteKit, que organiza la aplicación en torno a un sistema de enrutamiento basado en el sistema de ficheros: cada carpeta dentro de `src/routes/` define una ruta, y los ficheros con nombres especiales (`+page.svelte`, `+layout.svelte`, `+page.server.ts`) tienen roles concretos dentro del ciclo de vida de la página. La Tabla 4 recoge las rutas de la aplicación.

Ruta	Descripción
/	Página de inicio: <i>hero</i> de bienvenida para usuarios no autenticados; lista de asignaturas y recursos para usuarios autenticados.
/search	Búsqueda global de recursos mediante Full Text Search.
/create	Formulario de subida de un nuevo recurso con sus archivos.
/user/[slug]	Perfil público de un usuario y sus recursos compartidos.
/auth	Pantalla de inicio de sesión con Google OAuth2.
/admin	Panel de administración para gestionar reportes (solo administradores).

Tabla 4: Rutas de la aplicación frontend.

El fichero `+layout.svelte` de la raíz actúa como *shell* de toda la aplicación y define dos disposiciones de navegación completamente distintas según el dispositivo. En escritorio se muestra una barra de navegación superior clásica con los enlaces principales centrados y el perfil de usuario a la derecha. En móvil esta barra desaparece y es sustituida por una barra de navegación fija en la parte inferior de la pantalla, siguiendo el patrón habitual en aplicaciones móviles, acompañada de un botón de acción flotante (FAB) para subir un recurso. Esta separación es un cambio de paradigma de navegación adaptado a cada contexto de uso. La interfaz también se adapta en función del estado de autenticación expuesto por `data.me`. La ruta `/auth` queda excluida de este shell mediante su propio `+layout.svelte` independiente, de forma que la pantalla de inicio de sesión se renderiza sin navegación.

Las páginas que necesitan datos del backend utilizan un fichero `+page.server.ts` con una función `load`, que SvelteKit ejecuta en el servidor antes de renderizar la página. La función accede a las cookies de sesión y al resultado del `load` del layout padre mediante `parent()`, evitando repetir la llamada de autenticación en cada página y garantizando que la primera carga llegue al navegador ya con los datos.

3.2.2 Backend

El backend es un servidor HTTP escrito en Go que expone la API REST en el puerto 8080. Está organizado en paquetes por dominio funcional, cada uno con su propio *handler*. Todos los *handlers* reciben una instancia del struct `App`, que agrupa las tres dependencias compartidas del sistema: el cliente de base de datos (`Queries`, generado por `sqlc`), el *pool* de conexiones a PostgreSQL (`DB`, para transacciones) y el cliente S3 (`Storage`). Este patrón evita la necesidad de un framework de inyección de dependencias, manteniendo el control explícito sobre las dependencias sin añadir capas de abstracción innecesarias.

```

1 type App struct {
2     DB      *pgxpool.Pool
3     Queries QuerierWithTx
4     Storage *storage.S3Client
5 }

```

go

Código 1: Struct App que agrupa las dependencias compartidas del backend.

Los paquetes del backend son los siguientes:

Paquete	Descripción
auth	Implementa el flujo OAuth2 con Google, la generación y validación de tokens JWT y el middleware de autenticación. También expone el middleware AdminOnly para restringir el acceso a rutas administrativas.
resources	Gestiona el ciclo de vida completo de los recursos: creación con subida de archivos a S3, consulta, descarga (tanto del paquete completo como de ficheros individuales), eliminación y sistema de reportes.
users	Expone el perfil del usuario autenticado y permite actualizar su universidad, estudio y asignaturas fijadas.
studies universities	Proporcionan endpoints de consulta y búsqueda por texto sobre estudios y universidades.
admin	Rutas protegidas exclusivamente para administradores que permiten revisar y gestionar los reportes de recursos.
storage	Encapsula el cliente minio-go para interactuar con cualquier proveedor S3-compatible, configurable mediante variables de entorno.
db/generated	Código Go generado automáticamente por sqlc a partir de las consultas SQL escritas a mano en el esquema.

Tabla 5: Paquetes del backend y su responsabilidad.

3.2.3 Autenticación

La autenticación se delega completamente a Google OAuth2 por lo que el usuario no crea credenciales propias en Nemsy, sino que inicia sesión con su cuenta de Google. El flujo completo se ilustra en la Figura 11.

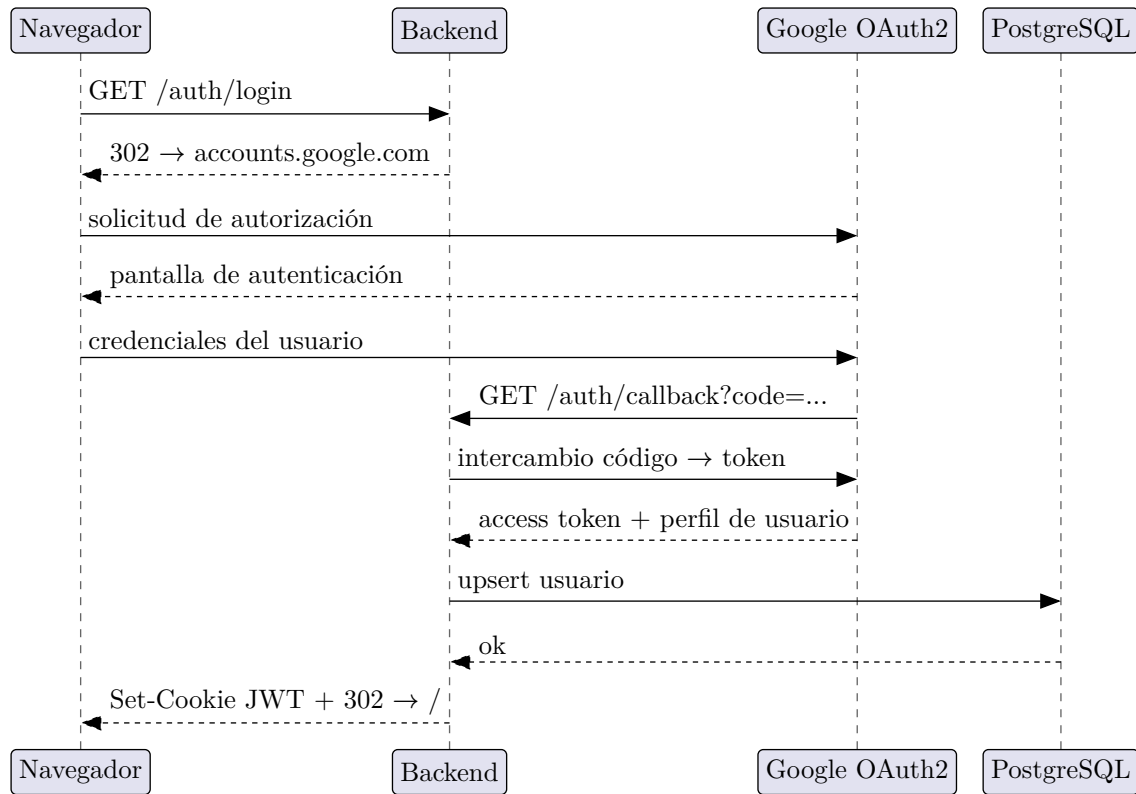


Figura 11: Diagrama de secuencia del flujo de autenticación con Google OAuth2.

Una vez completado el flujo, las rutas protegidas pasan por el middleware `AuthMiddleware`, que valida la cookie JWT en cada petición. La detección del centro educativo se realiza a partir del dominio del correo, si el usuario se autentica con una cuenta `@ucm.es`, se asocia automáticamente a la Universidad Complutense de Madrid sin necesidad de seleccionarla manualmente.

Esta separación se refleja directamente en la estructura del router ya que Chi permite anidar grupos de rutas con middlewares independientes, de forma que las rutas públicas de autenticación, las rutas protegidas por JWT y las rutas exclusivas de administración quedan aisladas entre sí.

3.2.4 Despliegue

El diagrama de la Figura 10 describe la arquitectura lógica del sistema, pero no cómo se materializa en el servidor. En producción, los tres componentes alojados en el VPS (frontend, backend y base de datos) corren cada uno en su propio contenedor Docker, orquestados por un único fichero `docker-compose.yml`. La Figura 12 ilustra esta disposición.

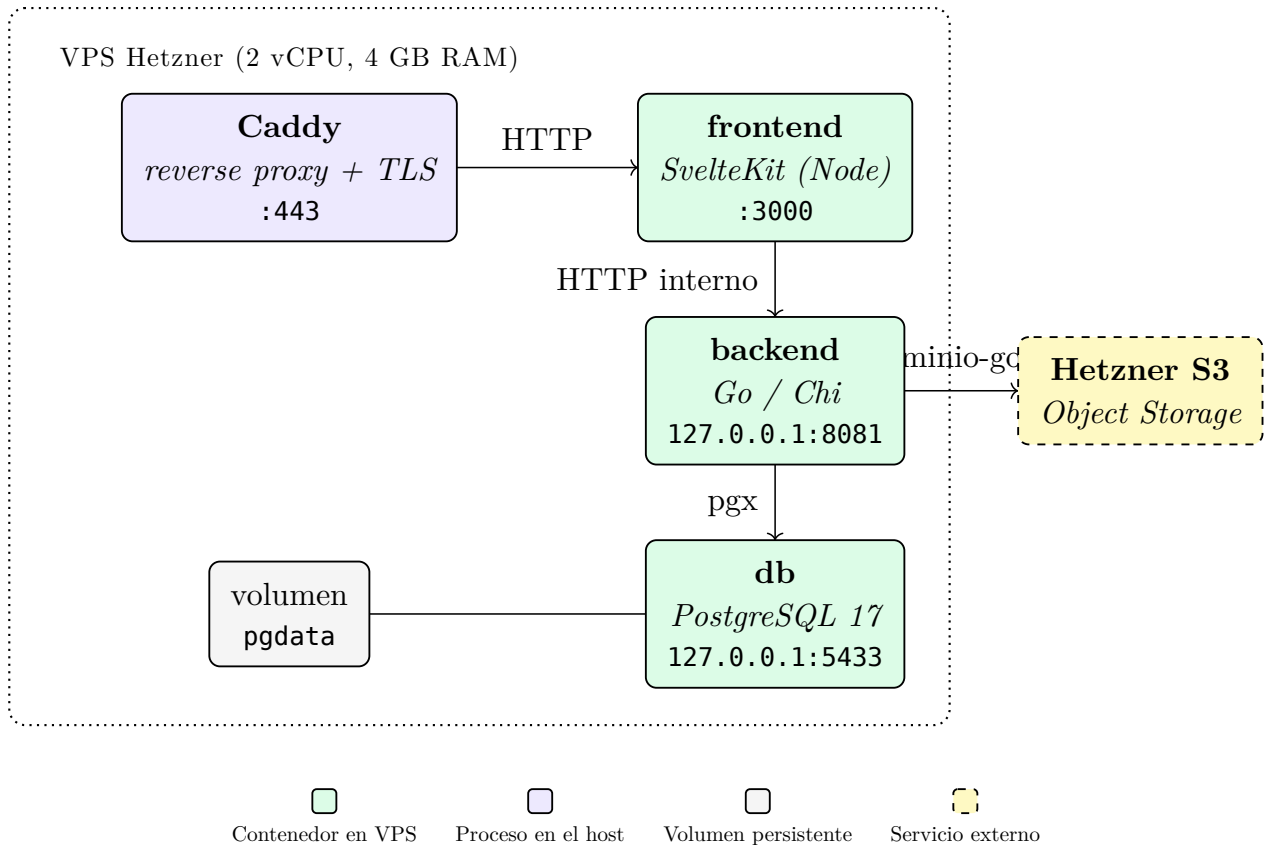


Figura 12: Arquitectura de despliegue en el VPS con Docker Compose.

Los tres servicios comparten una red interna gestionada por Docker, de forma que se comunican entre sí mediante el nombre del servicio (**db**, **backend**) sin exponer esos puertos al exterior. Los puertos del backend y la base de datos se publican únicamente en la interfaz de *loopback* (127.0.0.1:8081 y 127.0.0.1:5433), accesibles desde el propio VPS para tareas de administración pero no desde Internet. El tráfico HTTPS público llega al frontend a través de Caddy [11], un reverse proxy instalado en el host que termina TLS y reenvía las peticiones al contenedor correspondiente. Caddy se eligió frente a alternativas como Nginx por su gestión automática de certificados TLS mediante Let's Encrypt sin configuración adicional, lo que permite renovar los certificados de forma transparente sin intervención manual.

El estado persistente de PostgreSQL se almacena en un volumen nombrado (**pgdata**), desacoplado del ciclo de vida del contenedor, de forma que se pueden recrear los contenedores sin perder los datos. Los archivos subidos por los usuarios no se guardan en el VPS, sino en Hetzner Object Storage a través de la librería **minio-go**, lo que mantiene la máquina prácticamente sin estado.

Toda la configuración sensible (credenciales de base de datos, secreto JWT, claves OAuth2 y S3) se inyecta a los contenedores mediante variables de entorno definidas en un fichero **.env** presente únicamente en el servidor. De este modo, el mismo **docker-compose.yml** sirve tanto para desarrollo local como para producción, cambiando solo el contenido del **.env**, y el entorno resulta reproducible con un único comando.

```
1  services:
2    db:
3      image: postgres:17-alpine
4      restart: unless-stopped
5      environment:
6        POSTGRES_USER: nemsy
7        POSTGRES_PASSWORD: ${DB_PASSWORD}
8        POSTGRES_DB: nemsy
9      volumes:
10       - pgdata:/var/lib/postgresql/data
11      ports:
12       - "127.0.0.1:5433:5432"
13
14    backend:
15      build: ./backend
16      restart: unless-stopped
17      depends_on: [db]
18      environment:
19        DATABASE_URL: postgresql://nemsy:${DB_PASSWORD}@db:5432/nemsy?
20        sslmode=disable
21        JWT_SECRET: ${JWT_SECRET}
22        # ... OAuth2 y credenciales S3
23      ports:
24       - "127.0.0.1:8081:8080"
25
26    frontend:
27      build:
28        context: ./frontend
29      args:
30        PUBLIC_API_BASE_URL: ${PUBLIC_API_BASE_URL}
31      restart: unless-stopped
32      depends_on: [backend]
33      ports:
34       - "3000:3000"
35
36    volumes:
37      pgdata:
```

Código 2: Fragmento del docker-compose.yml de Nemsy.

3.3 Diseño de la interfaz de usuario

La interfaz de Nemsy se ha diseñado en paralelo al desarrollo del backend, partiendo de una primera versión funcional pero poco refinada que ha ido madurando hasta el lenguaje visual actual. En esta sección se recoge esa evolución y los tres pilares que sostienen el diseño final, el sistema de componentes, la tipografía y la paleta de color, que en conjunto buscan ofrecer una interfaz sobria pero no fría, alineada con el objetivo de competir en experiencia de usuario con plataformas como Wuolah.

3.3.1 Evolución del diseño

El diseño de Nemsy evolucionó de forma iterativa a lo largo del desarrollo. La primera versión, visible en la Figura 13, ya establecía la estructura que se mantiene hoy, con islas blancas sobre fondo de color y retícula de tres columnas, pero el lenguaje visual era más informal, con bordes negros gruesos, esquinas redondeadas y fondo azul pálido. A medida que se añadían nuevas pantallas, el diseño fue derivando hacia una estética más sobria, sustituyendo los bordes gruesos por líneas finas, las esquinas redondeadas por rectas y el azul pálido por un gris neutro. La Figura 14 muestra el estado actual en los dos modos de visualización disponibles.

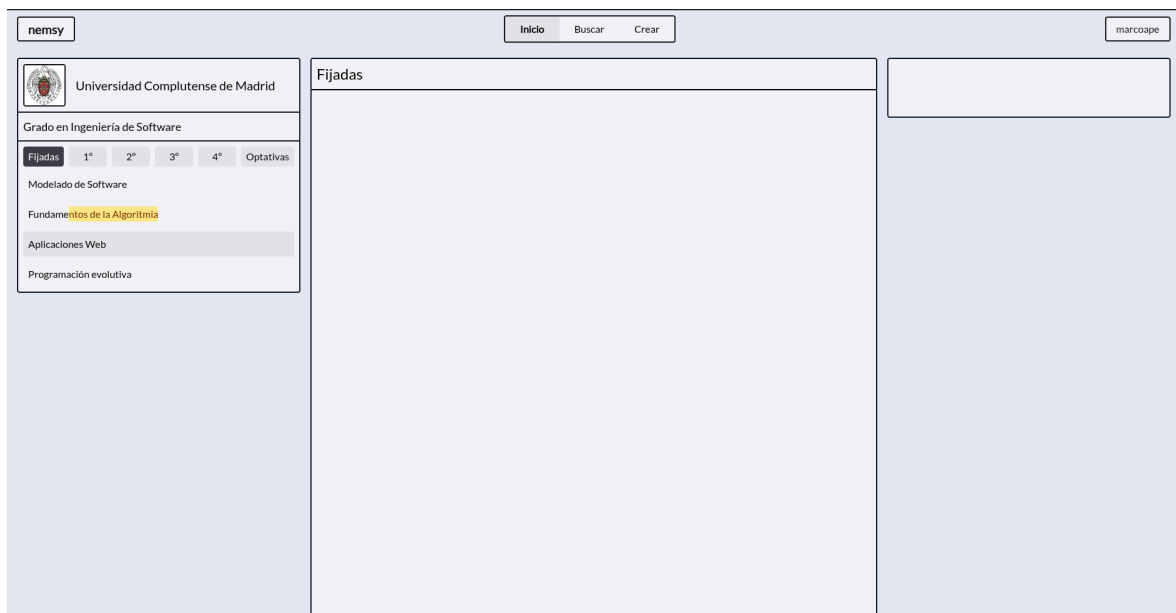


Figura 13: Primera versión de la interfaz de Nemsy (enero 2026).

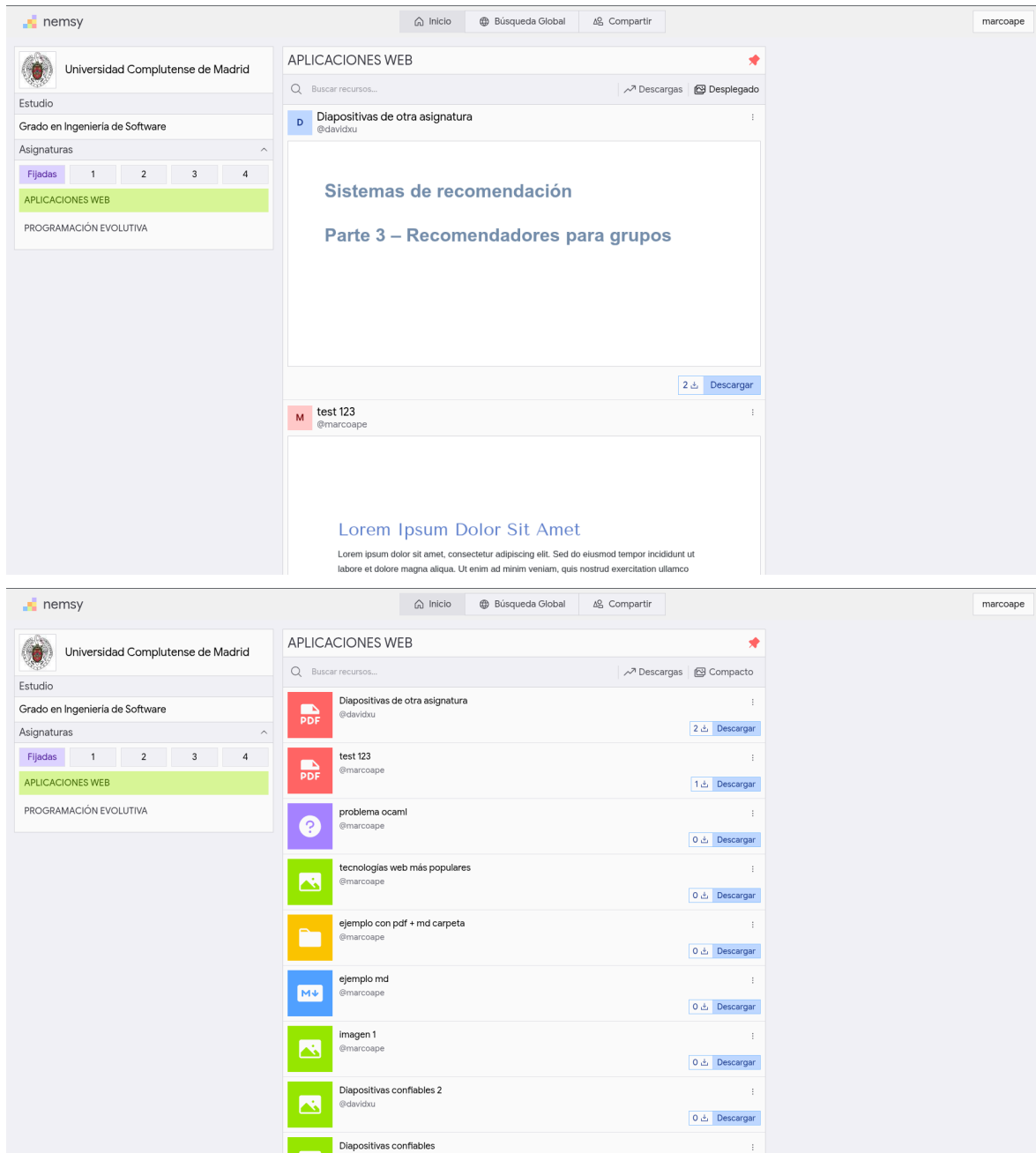


Figura 14: Interfaz actual de Nemsy en modo desplegado (arriba) y compacto (abajo).

3.3.2 Sistema de diseño

La interfaz organiza el contenido en islas blancas sobre un fondo gris claro, con esquinas rectas y bordes finos, buscando transmitir una sensación más ordenada y sobria que la primera versión. El logotipo, mostrado en la Figura 15, es el elemento visualmente más llamativo y resume la identidad visual de la plataforma: seis cuadrados de colores dispuestos sobre una retícula de 3×3 que forman una escalera ascendente junto al nombre. Esos seis colores no se quedan en el logotipo, sino que se reutilizan como acentos a lo largo de toda la interfaz, lo que evita que la rigidez de la cuadrícula y los bordes rectos resulten monótonos.

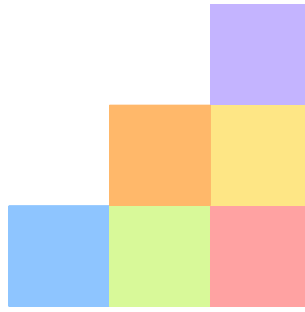


Figura 15: Logotipo de Nemsy.

3.3.3 Tipografía

La tipografía es *Google Sans Flex*, una fuente variable cargada desde Google Fonts que cubre todos los pesos y tamaños ópticos en un único fichero, evitando así la carga de múltiples variantes. Se eligió por su buena legibilidad a tamaños pequeños y porque sus formas ligeramente redondeadas contrastan con la rectitud de los componentes sin que la interfaz resulte fría.

3.3.4 Paleta de color

La paleta se divide en dos grupos. Por un lado, los colores base, una escala neutra de grises (zinc de Tailwind CSS) que estructura el fondo, las islas de contenido, los bordes y la jerarquía tipográfica. Por otro, los seis colores de acento heredados directamente del logotipo, también tomados de la paleta de Tailwind, que se aplican en pequeños detalles como fondos de elementos seleccionados, botones de acción e iconos de tipo de archivo. A diferencia del enfoque habitual de elegir un único color de acento, este reparto multicolor consigue que la interfaz, pese a su rigidez geométrica y su sobriedad, mantenga el suficiente contraste cromático como para no resultar aburrida de mirar ni de usar. La Figura 17 ilustra esta paleta en la página de búsqueda global.



Figura 16: Paleta de color de Nemsy basada en colores por defecto de Tailwind CSS.

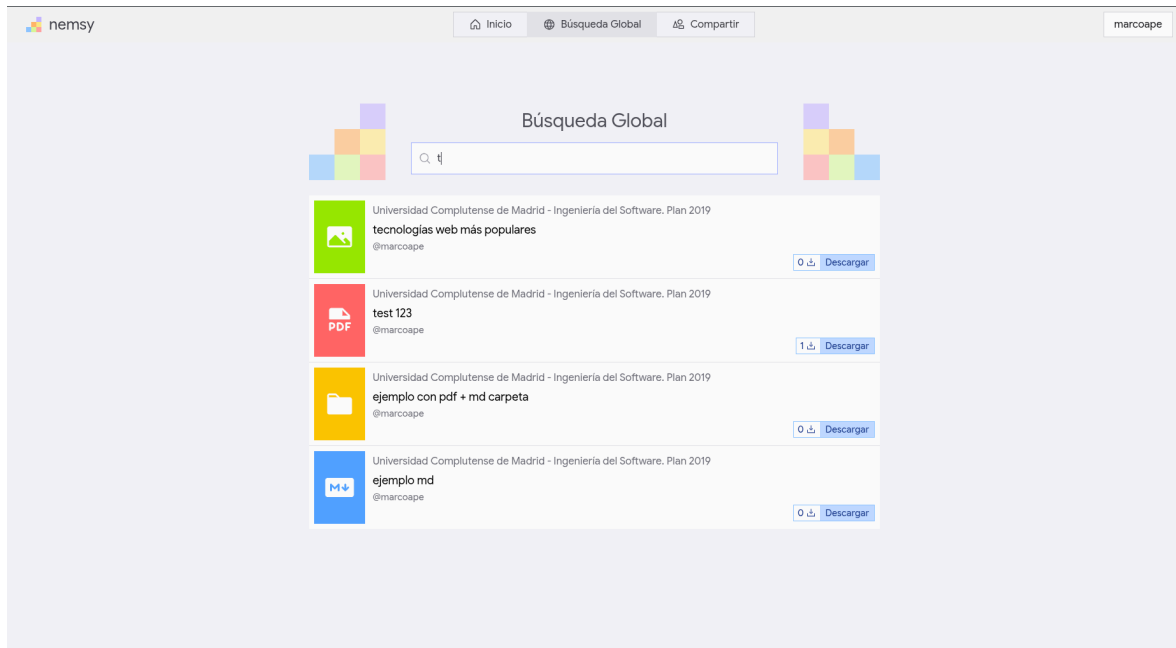


Figura 17: Página de búsqueda global de Nemsy.

3.4 Diseño de la base de datos

El esquema de Nemsy se apoya en nueve tablas relacionales que modelan las entidades principales de la plataforma y las relaciones entre ellas. El diseño sigue la tercera forma normal en lo que respecta a la información de dominio, pero introduce de forma deliberada dos columnas desnormalizadas (`download_count` y `search_vector`) para evitar cálculos repetidos en cada consulta. La Figura 18 recoge el modelo entidad-relación resultante.

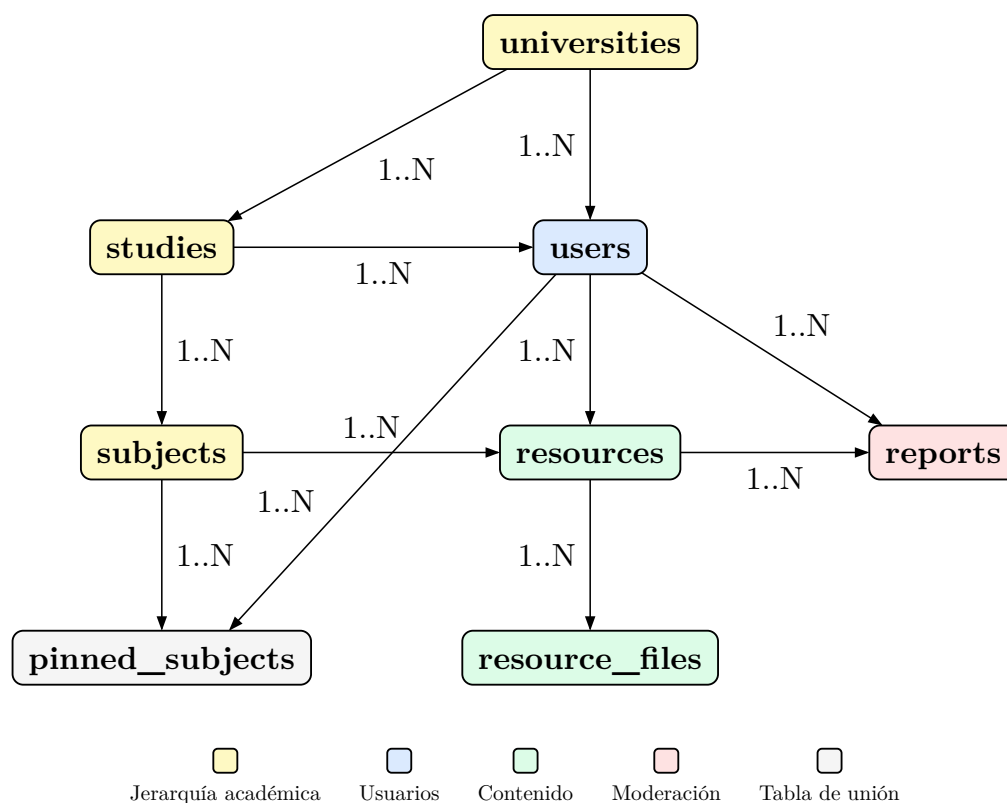


Figura 18: Diagrama entidad-relación del esquema de Nemsy.

3.4.1 Entidades principales

El esquema se articula en torno al eje educativo `universities` → `studies` → `subjects`, que modela la jerarquía académica, y al eje de contenido `users` → `resources` → `resource_files`, que modela la autoría y el almacenamiento. La tabla `pinned_subjects` materializa la relación N:M entre usuarios y asignaturas fijadas, y `reports` recoge las denuncias sobre recursos. La Tabla 6 resume el papel de cada tabla.

Tabla	Responsabilidad
<code>universities</code>	Catálogo de universidades con su dominio de correo (<code>ucm.es</code> , <code>upm.es</code> , ...), usado para asociar automáticamente al usuario al iniciar sesión con Google.
<code>studies</code>	Grados o titulaciones ofertados por una universidad. Cada estudio pertenece a una universidad mediante <code>university_id</code> .
<code>subjects</code>	Asignaturas que componen un estudio, junto con el curso (<code>year</code>) al que pertenecen.
<code>users</code>	Usuarios autenticados vía Google. Se almacena <code>google_sub</code> como identificador estable, el <code>email</code> , el <code>hd</code> (<i>hosted domain</i>) del que se deduce la universidad, un <code>username</code> único generado a partir del correo y un campo <code>role</code> para distinguir administradores.
<code>resources</code>	Publicaciones subidas por los usuarios. Cada recurso agrupa uno o varios archivos bajo un mismo <code>title</code> y <code>description</code> , y mantiene un contador <code>download_count</code> y un <code>search_vector</code> para la búsqueda de texto completo.
<code>resource_files</code>	Archivos individuales que componen un recurso, con su <code>s3_key</code> (clave en Object Storage), nombre original y tamaño.
<code>pinned_subjects</code>	Relación N:M entre <code>users</code> y <code>subjects</code> que representa las asignaturas fijadas por cada usuario en su página de inicio. Su clave primaria compuesta (<code>user_id</code> , <code>subject_id</code>) evita duplicados sin un índice adicional.
<code>reports</code>	Denuncias sobre recursos. La restricción <code>UNIQUE(resource_id, reporter_id)</code> impide que un mismo usuario denuncie dos veces el mismo recurso.

Tabla 6: Tablas del esquema y su responsabilidad.

Todas las claves foráneas se declaran con `ON DELETE CASCADE`, de forma que al eliminar un usuario se borran en cascada sus recursos, archivos, denuncias y asignaturas fijadas, sin dejar filas huérfanas ni necesidad de lógica de limpieza en el backend.

3.4.2 Evolución mediante migraciones

El esquema no se definió en un único fichero monolítico, sino que se construyó de forma incremental a medida que crecieron los requisitos de la plataforma. Para gestionar esta evolución se utilizó `golang-migrate` [12], una herramienta que aplica migraciones SQL numeradas y mantiene una tabla `schema_migrations` interna para saber qué versión está activa. Cada migración consta de dos ficheros, `NNN_nombre.up.sql` para aplicar el cambio y `NNN_nombre.down.sql` para revertirlo, lo que permite volver a una versión anterior sin intervención manual sobre la base de datos.

La Tabla 7 recoge el historial completo de migraciones del proyecto, que refleja la evolución natural del esquema desde la versión inicial hasta el estado actual.

Migración	Cambio introducido
000001_init	Esquema inicial con <code>studies</code> , <code>users</code> , <code>subjects</code> y <code>resources</code> .
000002_multi_files	Extrae los archivos de <code>resources</code> a la tabla <code>resource_files</code> para permitir varios ficheros por recurso.
000003_pinned_subjects	Tabla <code>pinned_subjects</code> para asignaturas fijadas por cada usuario.
000004_username	Sustituye <code>full_name</code> y <code>pfp</code> por un <code>username</code> único generado del correo.
000005_download_count	Añade <code>download_count</code> a <code>resources</code> para evitar <code>COUNT(*)</code> en cada consulta.
000006_search_vector	Añade la columna generada <code>search_vector</code> e índice GIN para Full Text Search.
000007_universities	Introduce la tabla <code>universities</code> y enlaza <code>studies</code> y <code>users</code> con ella.
000008_unaccent_search	Integra la extensión <code>unaccent</code> en los <code>search_vector</code> para ignorar tildes.
000009_user_role	Añade el campo <code>role</code> a <code>users</code> y la tabla <code>reports</code> .

Tabla 7: Historial de migraciones del esquema.

3.4.3 Búsqueda de texto completo

Una de las decisiones de diseño más relevantes es el uso de las capacidades nativas de Full Text Search de PostgreSQL para implementar la búsqueda global de recursos, sin recurrir a servicios externos como Elasticsearch. La búsqueda se apoya en tres piezas que se combinan en el propio esquema: columnas generadas de tipo `tsvector`, la extensión `unaccent` envuelta en una función `IMMUTABLE`, e índices GIN.

La columna `search_vector` de `resources` es una columna generada y almacenada (`GENERATED ALWAYS ... STORED`) que PostgreSQL recalcula automáticamente en cada `INSERT` o `UPDATE`, eliminando la necesidad de triggers o lógica adicional en el backend. Su contenido combina el título y la descripción del recurso con pesos distintos ('A' y 'B'), de forma que una coincidencia en el título pesa más que una en la descripción al calcular el ranking con `ts_rank`.

Un problema importante que surge al tratar con texto en español son las tildes, que pese a ser frecuentes en el idioma, los usuarios no siempre tienden a escribirlas al buscar pero si esperan que la búsqueda lo encuentre. La extensión `unaccent` de PostgreSQL elimina los diacríticos, pero no puede usarse directamente dentro de una columna generada porque no está marcada como `IMMUTABLE`. La solución, introducida en la migración `000008`, consiste en envolverla en una función SQL propia (`immutable_unaccent`) que sí lo está, tal y como muestra el fragmento de la Código 3.

```

1 CREATE OR REPLACE FUNCTION immutable_unaccent(text)
2 RETURNS text AS $$
3     SELECT public.unaccent('public.unaccent', $1)
4 $$ LANGUAGE sql IMMUTABLE PARALLEL SAFE;
5
6 ALTER TABLE resources ADD COLUMN search_vector tsvector
7     GENERATED ALWAYS AS (
8         setweight(to_tsvector('spanish',
9             immutable_unaccent(coalesce(title, ''))), 'A') ||
10        setweight(to_tsvector('spanish',
11            immutable_unaccent(coalesce(description, ''))), 'B')
12     ) STORED;
13
14 CREATE INDEX idx_resources_search ON resources USING GIN
    (search_vector);

```

Código 3: Definición del `search_vector` con pesos y `unaccent`.

Sobre esta columna se construye un índice GIN (*Generalized Inverted Index*), la estructura recomendada por PostgreSQL para datos `tsvector` [13]. Un índice GIN indexa cada *lexema* de forma independiente, de manera que una consulta con el operador `@@` localiza los recursos coincidentes sin necesidad de escanear toda la tabla. La misma técnica se aplica a la tabla `universities`, aunque en su caso se usa la configuración `'simple'` en lugar de `'spanish'`, ya que los nombres propios no deben reducirse a su raíz como sí conviene hacer con el texto en prosa.

3.4.4 Índices adicionales

Además del índice GIN para la búsqueda, el esquema define varios índices B-tree sobre las claves foráneas más consultadas. El índice compuesto `idx_resources_subject_created` (`subject_id`, `created_at` DESC) es especialmente relevante ya que sirve para la consulta que lista los recursos de una asignatura ordenados por fecha, permitiendo que PostgreSQL resuelva la operación directamente desde el índice sin ordenación posterior. Los índices por `owner_id` y por `resource_id` en `resource_files` garantizan que las eliminaciones en cascada y las consultas de perfil se ejecuten en tiempo logarítmico, sin necesidad de recorrer la tabla completa.

3.5 Implementación del backend

Detalles de implementación del backend en Go...

```
1 // rutas públicas (sin autenticación)
2 r.Get("/auth/login", authHandler.LoginHandler)
3 r.Get("/auth/callback", authHandler.CallbackHandler)
4 r.Post("/auth/logout", authHandler.LogoutHandler)
5
6 // rutas protegidas (JWT validado en cada petición)
7 r.Group(func(r chi.Router) {
8     r.Use(mw.Middleware)
9     r.Get("/api/me", usersHandler.MeHandler)
10    // ...
11
12    // solo administradores
13    r.Group(func(r chi.Router) {
14        r.Use(auth.AdminOnly)
15        r.Get("/api/admin/reports", adminHandler.ListReports)
16    })
17 })
```

Código 4: Estructura de grupos de rutas en Chi con middlewares anidados.

3.6 Implementación del frontend

Detalles de implementación del frontend en SvelteKit...

3.7 Pruebas y validación

Para garantizar la corrección y el rendimiento de la plataforma se aplicaron tres niveles de pruebas complementarios, cubriendo los tests unitarios del backend con el paquete `testing` de Go, los tests unitarios y *end to end* del frontend con Vitest y Playwright, y una prueba de carga con k6 sobre la API desplegada en producción. Esta última no solo valida que el sistema no falla bajo carga, sino que sirve como demostración empírica del rendimiento de la plataforma, aportando los datos que respaldan uno de sus objetivos centrales, ofrecer una experiencia rápida como alternativa a Wuolah.

3.7.1 Tests unitarios del backend

Los tests unitarios del backend están escritos con el paquete `testing` de la librería estándar de Go y `httptest` para simular peticiones HTTP sin levantar un servidor real. Cada paquete define un `mockQuerier` que implementa la interfaz `QuerierWithTx`

generada por sqlc, de forma que los tests son completamente independientes de la base de datos y se ejecutan de forma determinista.

Los tests del paquete `auth` cubren la generación y verificación de tokens JWT, la extracción de claims y el comportamiento del middleware ante distintos escenarios, incluyendo petición sin cookie, token con firma incorrecta y token válido. Este último caso verifica además que el middleware inyecta correctamente la información del usuario en el contexto de la petición.

```

1  func TestAuthMiddleware_ValidToken(t *testing.T) {
2      secret := []byte("testsecret")
3      tokenStr, _ := GenerateJWTWithUserID(
4          UserInfo{GoogleSub: "123456", Email: "test@example.com"},
5          42, "user", secret,
6      )
7
8      req := httptest.NewRequest("GET", "/protected", nil)
9      req.AddCookie(&http.Cookie{Name: "session_token", Value:
10         tokenStr})
11      rr := httptest.NewRecorder()
12
13      mw := &AuthMiddleware{Secret: secret}
14      handlerCalled := false
15
16      mw.Middleware(http.HandlerFunc(func(w http.ResponseWriter, r
17         *http.Request) {
18         handlerCalled = true
19         w.WriteHeader(http.StatusOK)
20     })).ServeHTTP(rr, req)
21
22     if rr.Code != http.StatusOK { t.Errorf("expected 200, got %d",
23         rr.Code) }
24     if !handlerCalled { t.Error("protected handler was not called") }
25 }

```

Código 5: Test del middleware de autenticación con token válido.

Los tests del paquete `search` verifican la función `PrefixQuery`, que transforma la entrada del usuario en una expresión compatible con el operador `@@` de PostgreSQL Full Text Search. Los casos incluyen búsquedas con una o varias palabras, eliminación de tildes y caracteres especiales, espacios extra y entrada vacía.

```

1  func TestPrefixQuery(t *testing.T) {
2      tests := []struct{ name, input, want string }{
3          {"single word", "matematicas", "matematicas:*"},
4          {"multiple words", "algebra lineal", "algebra:* & lineal:*"},
5          {"strips accents", "programación", "programacion:*"},
6          {"removes specials", "hello! world?", "hello:* & world:*"},
7          {"empty input", "", ""},
8          {"extra whitespace", "  fisica cuantica  ", "fisica:* & cuantica:*"},
9      }
10     for _, tt := range tests {
11         t.Run(tt.name, func(t *testing.T) {
12             got := PrefixQuery(tt.input)
13             if got != tt.want {
14                 t.Errorf("PrefixQuery(%q) = %q, want %q", tt.input, got, tt.want)
15             }
16         })
17     }
18 }

```

Código 6: Tests de la función PrefixQuery para Full Text Search.

Los tests de los *handlers* siguen el mismo patrón: instancian un `mockQuerier` con funciones anónimas que devuelven datos controlados, construyen una petición HTTP con `httptest.NewRequest`, inyectan el contexto de autenticación manualmente y verifican tanto el código de estado como el cuerpo de la respuesta.

3.7.2 Tests unitarios del frontend

Los tests unitarios del frontend se escribieron con Vitest [14] y se centran en los dos tipos de lógica que pueden probarse de forma aislada, los componentes de interfaz y las acciones de Svelte.

El componente `HighlightText` resalta la subcadena buscada dentro de un texto envolviéndola en un elemento `<mark>`. Sus tests verifican los tres casos posibles, que resalta correctamente ignorando mayúsculas y tildes, que no introduce ningún `<mark>` cuando no hay coincidencia, y que tampoco lo hace con una búsqueda vacía.


```

1  it('highlights the matching substring', async () => { typescript
2      render(HighlightText, { text: 'Álgebra Lineal', query:
        'lineal' });
3      const mark = page.getByRole('mark');
4      await expect.element(mark).toHaveTextContent('Lineal');
5  });
6
7  it('renders plain text when there is no match', async () => {
8      const { container } = render(HighlightText, { text: 'Cálculo',
        query: 'física' });
9      expect(container.querySelectorAll('mark').length).toBe(0);
10 });

```

Código 7: Tests unitarios del componente HighlightText.

La acción `clickOutside` detecta clics fuera de un elemento del DOM y emite un evento `outclick`, usado para cerrar menús desplegables. Sus tests verifican que el evento se dispara al hacer clic fuera del nodo, que no se dispara al hacer clic dentro, y que deja de escuchar tras llamar a `destroy`.

```

1  it('dispatches outclick when clicking outside the node', typescript
    () => {
2      const node = document.createElement('div');
3      const outside = document.createElement('div');
4      document.body.appendChild(node);
5      document.body.appendChild(outside);
6
7      let fired = false;
8      node.addEventListener('outclick', () => { fired = true; });
9
10     const action = clickOutside(node);
11     outside.click();
12
13     expect(fired).toBe(true);
14     action.destroy!();
15 });

```

Código 8: Test unitario de la acción `clickOutside`.

3.7.3 Tests end to end con Playwright

Los tests *end to end* (E2E) verifican los flujos de la aplicación desde el punto de vista del navegador, ejecutando interacciones reales contra el frontend y el backend levantados localmente. Se escribieron con Playwright [15], una herramienta de auto-

matización de navegadores que permite controlar Chrome, Firefox y Safari desde código TypeScript.

El principal reto de los tests E2E en Nemsy es que la autenticación delega en Google OAuth2, cuyo flujo no puede automatizarse en un entorno de pruebas. La solución adoptada es inyectar directamente una cookie `session_token` con un JWT firmado antes de cada test, usando un helper `generateTestJWT` implementado en TypeScript con la API de criptografía de Node.js. De este modo, los tests arrancan ya autenticados sin pasar por el flujo de Google.

```
1 test.beforeEach(async ({ context }) => { typescript
2     const jwt = generateTestJWT({
3         sub: 'e2e-test-sub', email: 'e2e@test.edu',
4         hd: 'test.edu', user_id: 32, role: 'user'
5     });
6     await context.addCookies([{
7         name: 'session_token', value: jwt,
8         domain: 'localhost', path: '/'
9     }]);
10 });
```

Código 9: Inyección de cookie JWT antes de cada test E2E.

Los tests están organizados en dos grupos. El grupo `logged out` verifica que la página de inicio muestra el *hero* y el botón de inicio de sesión cuando no hay sesión activa. El grupo `logged in` cubre los flujos principales, verificando que la página de inicio muestra la barra lateral de asignaturas, que la búsqueda global devuelve resultados al escribir, que el formulario de subida muestra todos sus campos y que el perfil de un usuario es accesible.

```
1 test('search returns results', async ({ page }) => { typescript
2     await page.goto('/search');
3     await page.waitForLoadState('networkidle');
4
5     const input = page.getByPlaceholder('Buscar recursos de toda la
6     plataforma...');
7     await input.click();
8     await input.pressSequentially('prueba', { delay: 50 });
9
10    await expect(page.getByText('Recurso de
11    prueba')).toBeVisible({ timeout: 10000 });
12 });
```

Código 10: Test E2E del flujo de búsqueda global.

como inmediata. Como se puede observar en la Figura 19 y en la Tabla 8, la API de Nemsy obtiene un p95 global de 3,45 ms bajo 100 VUs concurrentes, casi treinta veces por debajo de ese umbral, con una tasa de error del 0 %. Cabe destacar que estas latencias corresponden exclusivamente al servidor, el tiempo que percibe el usuario final incluye además la red, el renderizado del navegador y la ejecución del JavaScript del frontend.

El script de prueba se encuentra en `tests/k6/stress-test.js`. Para ejecutar las rutas protegidas, se incluyó un generador de tokens JWT en `backend/cmd/gen-token/main.go` que firma un token con el mismo secreto que el servidor, evitando la necesidad de pasar por el flujo de Google OAuth2 durante la prueba.

3.7.4.2 Análisis con Lighthouse

Para medir el rendimiento desde la perspectiva del usuario se utilizó Google Lighthouse, ejecutando la auditoría en las mismas condiciones para ambas plataformas mediante Helium [18], un fork ligero de Chromium sin extensiones instaladas y en modo incógnito, accediendo a la página de inicio tras iniciar sesión. El resultado se recoge en la Figura 20.

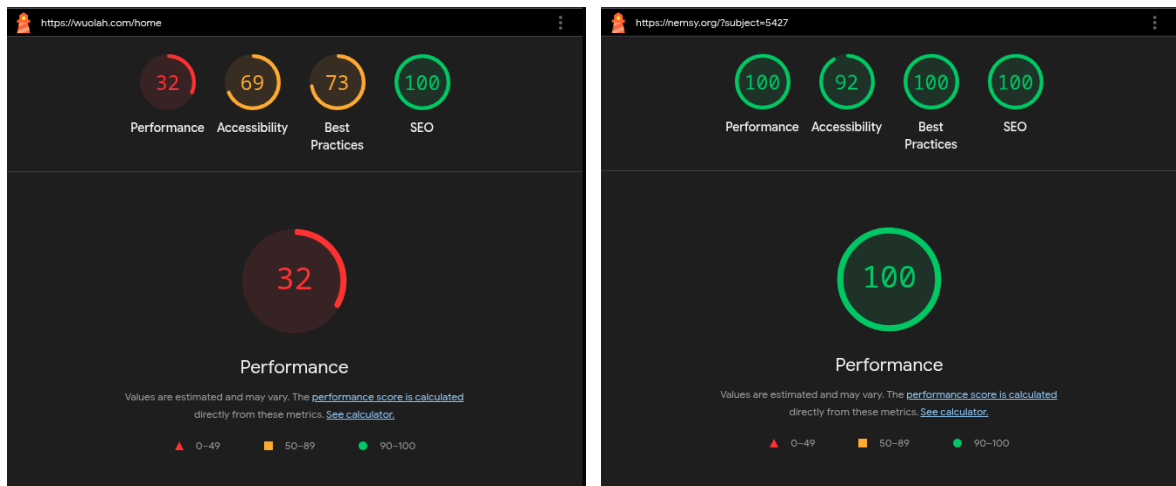


Figura 20: Comparativa de puntuaciones Lighthouse entre Wuolah (izquierda) y Nemsy (derecha).

Nemsy obtiene una puntuación de rendimiento de 100 sobre 100, frente al 32 de Wuolah. La diferencia responde principalmente a la ausencia de publicidad y scripts de terceros, que en Wuolah son la principal fuente de bloqueo del hilo principal del navegador, a un bundle de JavaScript mínimo gracias a que Svelte compila los componentes a JavaScript puro sin necesidad de ningún framework en tiempo de ejecución, y a una API en Go que, como demuestran los resultados de la sección anterior, responde en menos de 4 ms en el percentil 95 independientemente de la carga concurrente.

Capítulo 4 Conclusiones y Trabajo Futuro

Resumen: Este capítulo presenta las conclusiones extraídas del proyecto y las líneas de trabajo futuro.

4.1 Conclusiones

Conclusiones del proyecto...

4.2 Objetivos alcanzados

Evaluación de los objetivos propuestos...

4.3 Trabajo futuro

Posibles mejoras y extensiones...

Bibliografía

- [1] E. L. Deci, R. Koestner, y R. M. Ryan, «A meta-analytic review of experiments examining the effects of extrinsic rewards on intrinsic motivation», 1999.
- [2] B. S. Frey y R. Jegen, «Motivation Crowding Theory», 2001.
- [3] Stack Overflow, «Stack Overflow Developer Survey 2025». [En línea]. Disponible en: <https://survey.stackoverflow.co/2025/technology>
- [4] The Go Authors, «The Go Programming Language». [En línea]. Disponible en: <https://go.dev/>
- [5] go-chi contributors, «Chi: lightweight, idiomatic and composable router for building Go HTTP services». [En línea]. Disponible en: <https://go-chi.io/>
- [6] Meta Open Source, «React: The library for web and native user interfaces». [En línea]. Disponible en: <https://react.dev/>
- [7] Svelte Contributors, «Svelte: Cybernetically enhanced web apps». [En línea]. Disponible en: <https://svelte.dev/>
- [8] Oracle Corporation, «MySQL: The world's most popular open source database». [En línea]. Disponible en: <https://www.mysql.com/>
- [9] MongoDB, Inc., «MongoDB: The Developer Data Platform». [En línea]. Disponible en: <https://www.mongodb.com/>
- [10] The PostgreSQL Global Development Group, «PostgreSQL: The World's Most Advanced Open Source Relational Database». [En línea]. Disponible en: <https://www.postgresql.org/>
- [11] Caddy Web Server, «Caddy: The Ultimate Server with Automatic HTTPS». [En línea]. Disponible en: <https://caddyserver.com/>
- [12] golang-migrate contributors, «golang-migrate: Database migrations. CLI and Golang library.». [En línea]. Disponible en: <https://github.com/golang-migrate/migrate>
- [13] PostgreSQL Global Development Group, «GIN Indexes». [En línea]. Disponible en: <https://www.postgresql.org/docs/current/gin.html>
- [14] Vitest Team, «Vitest: Next Generation Testing Framework ». [En línea]. Disponible en: <https://vitest.dev/>

- [15] Microsoft, «Playwright: Playwright enables reliable web automation for testing, scripting, and AI agents.». [En línea]. Disponible en: <https://playwright.dev/>
- [16] Grafana Labs, «k6: The best developer experience for load testing». [En línea]. Disponible en: <https://k6.io/>
- [17] Google, «RAIL performance model». [En línea]. Disponible en: <https://web.dev/articles/rail>
- [18] inputnet, «Helium: Internet without interruptions». [En línea]. Disponible en: <https://helium.computer/>

